



POLITECHNIKA POZNAŃSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI
Instytut Informatyki

Praca dyplomowa magisterska

ZASTOSOWANIE DUŻYCH MODELI JĘZYKOWYCH DO TWORZENIA INTELIGENTNYCH AGENTÓW W GRACH KOMPUTEROWYCH

Marcin Korcz, 144446

Promotor
dr inż. Iwo Błądek

POZNAŃ 2024

Tutaj będzie karta pracy dyplomowej;
oryginał wstawiamy do wersji dla archiwum PP, w pozostałych kopiach wstawiamy ksero.

Spis treści

1	Wstęp	1
1.1	Motywacja podjęcia pracy	1
1.2	Cel i zakres pracy	2
2	Podstawy teoretyczne	4
2.1	Duże modele językowe	4
2.1.1	Uczenie w kontekście	5
2.1.2	Zdolności dużych modeli językowych w perswazji	7
2.2	Inteligentny agent	8
2.2.1	Klasyczne definicje inteligentnych agentów	8
	Środowisko	9
	Agent	11
2.2.2	Agent bazujący na dużym modelu językowym	12
	Prace wykorzystujące agentów bazujących na dużych modelach języka . . .	15
3	Środowisko	17
3.1	Właściwości środowiska	18
3.2	Implementacja	18
3.3	Przystosowanie do eksperymentów	19
4	Agent	20
4.1	Zadania agenta	20
4.2	Implementacja	20
4.2.1	Agent fazy akcji	20
4.2.2	Agent fazy dyskusji	21
4.2.3	Agent fazy głosowania	22
4.3	Ograniczenia	23
5	Przykładowy przebieg gry	24
5.1	Wnioski z przebiegu	25
5.1.1	Ograniczenia	26
6	Eksperymenty	28
6.1	Ocena agenta podczas etapu wykonywania akcji	28
6.1.1	Wyniki	28
6.2	Analiza gier z fazy dyskusji	29
6.3	Ograniczenia badań i potencjalne kierunki przyszłych prac	29
7	Podsumowanie	31

7.1	Wnioski	31
7.2	Możliwy kierunek przyszłych prac	32
	Literatura	33
	A Użyte prompty	36
	B Zapisy przebiegów eksperymentu	39

Rozdział 1

Wstęp

1.1 Motywacja podjęcia pracy

Na przestrzeni ostatnich lat gry komputerowe stały się nie tylko jedną z najpopularniejszych form rozrywki, ale też sektorem branży informatycznej, który potrzebuje i wytwarza przełomowe rozwiązania technologiczne. Dynamiczne postępy można zauważyć w wielu dziedzinach informatyki, w tym sztucznej inteligencji. Przykłady wykorzystania sztucznej inteligencji w grach są szerokie. Można zaliczyć do nich między innymi: *zwiększanie rozdzielczości* (ang. *upscaling*) [9], *proceduralne generowanie treści* (PCG, ang. *Procedural Content Generation*) [29], jak i tworzenie i zarządzanie *niegrywalnymi postaciami* (NPC, ang. *Non-player character*) [39]. Tworzenie i zarządzanie niegrywalnymi postaciami jest jedną z najstarszych zastosowań sztucznej inteligencji w grach komputerowych, gdzie pierwsze zastosowanie datowane jest na 1952 rok w grze Nim [30, 39] i od tego czasu jest praktycznie nierozłączną częścią gier komputerowych, która jest ciągle rozwijana. Kolejną ewolucję w tym aspekcie gier wideo możemy zanotować wraz z powstaniem gry Pac-Man, gdzie każdy z duchów goniących gracza ma własną osobowość i styl gry, co nadaje autentyczności oraz kolorytu rozgrywce [6]. Od tego czasu twórcy gier prześcigają się w tworzeniu coraz to lepszych, bardziej immersyjnych i wiarygodnych postaci w grach wideo dodając im więcej opisów, ścieżek dialogowych oraz mechanizmów reagowania na działania gracza, co ma pomóc upodobnić niegrywalne postaci do ludzi. Niestety w dalszym ciągu nie udaje się stworzyć sztucznej inteligencji zdolnej dostatecznie dobrze osiągać założone cele, która byłaby jednocześnie w swoim zachowaniu podobna do ludzkich graczy. Potencjalnym rozwiązaniem tego problemu może być postępująca rewolucja w sztucznej inteligencji, którą zapoczątkowały *duże modele językowe* (LLM, ang. *Large Language Models*).

Wraz z udostępnieniem przez firmę OpenAI narzędzia ChatGPT [27], nastąpił gwałtowny rozwój i zainteresowanie dużymi modelami języka, których to nowe zdolności są wciąż odkrywane. Jednym z praktycznych zastosowań dużych modeli języka jest utworzenie za ich pomocą inteligentnych agentów, którzy potrafią rozumować, tworzyć plany rozwiązania danego problemu, a potem go wykonać [32, 28]. Agenci ci są wykorzystywani jako osobiści asystenci w codziennym życiu lub pracy, na przykład: używanie jako wyszukiwarki informacji, wysokiej jakości tłumaczenie tekstu, streszczanie maili, odpowiadanie na maile, streszczenie i analiza raportów finansowych firm. Tacy agenci mogą okazać się skutecznymi w tworzeniu wiarygodnych i inteligentnych niegrywalnych postaci w grach komputerowych.

Nagle zainteresowanie świata nową technologią spowodowało powstanie konkurencyjnych rozwiązań do rodziny GPT od firmy OpenAI, takich jak modele Llama [24] od firmy Meta, modeli Claude [4] od firmy Anthropic oraz wielu innych rodzin dużych modeli języka. Zarówno mnogość

rozwiązań i ich podobieństwo do siebie, jak i problemy z wiarygodnymi miarami jakości modeli, tworzą zapotrzebowanie na nowe i zróżnicowane testy zdolności danych modeli. Próba stworzenia rankingu modeli językowych za pomocą środowisk gier wideo wydaje się obiecująca, gdyż miarą może być wygranie gry bądź szybkie rozwiązanie danego problemu. Uzyskana w ten sposób miara jest prosta w interpretacji, a dodatkowo środowisko to mogłoby posłużyć do uczenia ze wzmocnieniem modeli, czyniąc ich zdolności rozumowania i perswazji jeszcze lepszymi.

Motywacją do podjęcia niniejszej pracy jest chęć zbadania, jak rozwój dużych modeli językowych może przyczynić się do dalszego rozwoju w tworzeniu inteligentnych niegrywalnych postaci. Wspomniani inteligentni agenci wykorzystujący duże modele językowe wydają się idealni do rozwiązania trudności, z którymi aktualnie zmagają się twórcy gier. Dodatkowo utworzone środowisko może posłużyć do dalszych prac nad udoskonalaniem samych dużych modeli języka.

1.2 Cel i zakres pracy

Kierując się motywacjami przedstawionymi w poprzedniej sekcji, celem pracy jest stworzenie agentów będących niegrywalnymi postaciami, którzy są zdolni dynamicznie reagować na działania gracza i innych agentów, prowadząc zbliżone do ludzkich interakcje i zapewniając wiarygodne doświadczenia w grze wzorowanej na grze o tytule Among Us [17]. Praca ma na celu analizę potencjału tych modeli, ocenę ich efektywności w kontekście gier oraz identyfikację wyzwań związanych z ich implementacją.

Wybór tej gry jako środowiska jest podyktowany złożonością i rodzajem zadań, które są powierzone każdemu z graczy. Among Us to odmiana gry Mafia[11], którą można opisać jako towarzyską grę fabularną symulującą walkę pomiędzy poinformowaną mniejszością i niepoinformowaną większością. Gra wymaga i rozwija umiejętności obserwacji i analizy zachowań, wyciągania wniosków, przekonywania oraz planowania. Rozgrywkę można podzielić na dwie części: fazę wykonywania akcji oraz fazę dyskusji, co posłuży nam do oceny jakości modeli, a będzie dokładniej opisane w rozdziale poświęconym środowisku.

Ze względu na ograniczenia sprzętowe związane z trudnościami z wczytaniem wag dużych modeli językowych, które potrafią zaalokować nawet 56 GB pamięci, oraz sporym nakładem kosztów obliczeniowych, w pracy będę korzystał tylko z modeli dostępnych poprzez API, a dokładniej modeli z rodziny GPT. Wiąże się to z ograniczonym budżetem na eksperymenty.

Pierwotnie praca miała skupiać się jedynie na tworzeniu i ocenie stworzonych inteligentnych agentów wcielających się w postaci niegrywalne. Ze względu na trudności i komplikacje w korzystaniu z Among-us [23] (otwartoźródłowej implementacji mechanik Among Us w trybie gry tekstowej zamiast oryginalnego interfejsu graficznego), zaistniała dodatkowo potrzeba stworzenia odpowiedniego środowiska dla inteligentnych agentów. Przeszkody wynikały z implementacji tej gry oraz z faktu, że wiele jej mechanik było tworzonych z myślą o ludzkim użytkowniku. Dalsza analiza konieczności zmiany podejścia będzie opisana w rozdziale poświęconym środowisku. Ze względu na te perturbacje, cele i zakres pracy zostały rozszerzone o stworzenie środowiska dostosowanego pod potrzeby testowania inteligentnych agentów i możliwości interakcji zarówno między sobą, jak i z ludźmi.

Struktura pracy jest następująca. W rozdziale 2 przedstawiono przegląd literatury na temat dużych modeli języka, promptingu, inteligentnych agentów oraz podobnych istniejących prac poruszających tematykę inteligentnych agentów wykorzystujących duże modele językowe. Rozdział 3 poświęcony jest opisowi środowiska. W tym rozdziale przedstawione zostaną reguły i przebieg gry oraz argumentacja wyboru tej gry. Opisany też będzie wybór użytych technologii i standardów, które mają posłużyć do utrzymywaności środowiska i jego rozwoju. Rozdział 4 porusza zagadnie-

nia związane z implementacją i zadaniami stawianymi przed inteligentnym agentem. Wymienione zostaną role, które agent może przyjąć w środowisku oraz konkretne zadania w danej roli. Zostanie tam też przedstawiony prompt użyty dla danego agenta w danym stanie wraz z uzasadnieniem jego użycia. Rozdział 5 przedstawia opis rozgrywki przeprowadzonej przez utworzonych agentów. Rozdział 6 poświęcony jest przeprowadzonym eksperymentom. Oprócz przedstawienia warunków początkowych, dla każdego z eksperymentów zostanie również przedstawiony jego przebieg. W eksperymentach będą oceniane poczynania inteligentnego agenta w fazie wykonywania akcji gry, oraz subiektywna ocena radzenia sobie w fazie dyskusji. Na koniec zostaną przedstawione propozycje dalszych eksperymentów oraz benchmarku dla modeli języka z wykorzystaniem tego środowiska. Rozdział 7 stanowi podsumowanie pracy magisterskiej.

Rozdział 2

Podstawy teoretyczne

Rozdział ten zawiera wprowadzenie pojęć i teorii kluczowych w zrozumieniu dalszych rozważań w niniejszej pracy. Przedstawione zostaną aktualnie używane techniki oraz pojęcia dotyczące dużych modeli językowych, jak i inteligentnych agentów. Na koniec przedstawione zostaną prace z literatury, które są powiązane z tematyką tej pracy.

2.1 Duże modele językowe

W ostatnich latach obserwuje się rewolucję w dziedzinie przetwarzania języka naturalnego (*NLP*, ang. *Natural Language Processing*), której katalizatorem stały się duże modele językowe (*LLM*, ang. *Large Language Models*). Te zaawansowane narzędzia oparte na uczeniu maszynowym zmieniły sposób, w jaki maszyny przetwarzają i generują język ludzki, otwierając nowe możliwości w zakresie interakcji człowiek-komputer.

Rozwój modeli językowych można prześledzić od prostych modeli statystycznych, przez rekurencyjne sieci neuronowe, aż po obecne architektury sieci neuronowych oparte na transformerach, wprowadzone w przełomowej pracy „Attention Is All You Need” w 2017 roku [35]. Duże modele językowe to modele trenowane na ogromnych zbiorach danych tekstowych, obejmujących miliardy słów z różnorodnych źródeł. Modele te są trenowane głównie poprzez przewidywanie następnego tokenu lub maskowanie i odgadywanie tokenów w sekwencji. Współczesne duże modele językowe, po wstępnym wytrenowaniu, wykorzystują techniki *uczenia ze wzmocnieniem na podstawie informacji zwrotnej od człowieka* (*RLHF*, ang. *Reinforcement learning from human feedback*) [10]. Dzięki temu są w stanie rozumieć kontekst, generować spójne i sensowne odpowiedzi, a nawet wykonywać złożone zadania językowe bez konieczności dodatkowego uczenia się wykonywania tych zadań.

Najbardziej znane przykłady dużych modeli językowych to modele z rodziny *GPT* (ang. *Generative Pre-trained Transformer*) [27] stworzone przez OpenAI, *BERT* (ang. *Bidirectional Encoder Representations from Transformers*) [12] opracowany przez Google, *LLAMA* (ang. *Large Language Model Meta AI*) [24] od firmy Meta, czy Claude [4] od firmy Anthropic. Każdy z wymienionych modeli ma swoje unikalne cechy, ale wszystkie opierają się na podobnych zasadach działania.

LLM wykazują szerokie spektrum możliwości, takich jak:

- generowanie spójnego i kontekstowo odpowiedniego tekstu,
- odpowiadanie na pytania na podstawie dostarczonego kontekstu,
- wysokiej jakości tłumaczenie tekstu,
- streszczanie długich dokumentów,

- analizowanie sentymentu i emocji zawartych w tekście,
- generowanie kodu programistycznego na podstawie opisu,
- prowadzenie dialogu w sposób zbliżony do ludzkiego.

Ta wszechstronność sprawia, że duże modele językowe znajdują zastosowanie w wielu dziedzinach, od automatyzacji obsługi klienta, przez wspomaganie tworzenia treści, aż po zaawansowane systemy wspomagania decyzji w biznesie i nauce.

Jednakże, jako stosunkowo młoda i dynamicznie rozwijająca się dziedzina sztucznej inteligencji, LLM stoją przed szeregiem wyzwań:

- kwestie etyczne związane z potencjalnym niewłaściwym wykorzystaniem,
- problem stronniczości (ang. *bias*) wynikający z użytych danych treningowych,
- trudności w zapewnieniu prywatności danych użytych do treningu modelu,
- ogromne zapotrzebowanie na moc obliczeniową i energię,
- trudności w weryfikacji prawdziwości generowanych informacji (halucynacja wiedzy).

Obecne kierunki badań koncentrują się na zwiększaniu efektywności modeli, redukcji ich rozmiaru przy zachowaniu wydajności, poprawie zdolności rozumowania i wnioskowania oraz na próbach radzenia sobie z wyżej wymienionymi wyzwaniami.

2.1.1 Uczenie w kontekście

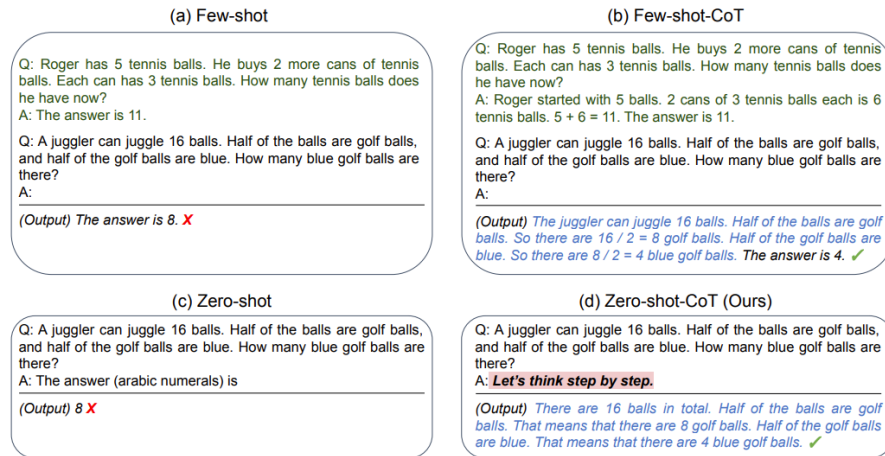
Uczenie w kontekście (ang. *in-context learning*) [7] to zjawisko obserwowane w dużych modelach językowych. Jest to zdolność modelu do adaptacji do nowych zadań bez konieczności aktualizacji wag połączeń, wyłącznie na podstawie informacji zawartych na wejściu modelu. Inaczej to przedstawiając, uczenie w kontekście to proces, w którym model dostosowuje swoją predykcję do sformułowanego zdania, jak i podanych przykładów jak model ma odpowiadać, w trakcie przechodzenia sygnału przez sieć. Zdolność ta nie jest bezpośrednim celem procesu uczenia modelu, ale emergentną właściwością dużych modeli językowych [36, 40]. Zdolność do uczenia w kontekście zależy od liczby parametrów modelu [19].

Prompting Sztuka formułowania zapytań lub instrukcji dla modelu językowego, mająca na celu uzyskanie pożądanego wyniku. W literaturze przedmiotu można spotkać różne określenia tego procesu: *prompting*, *prompt engineering* lub *inżynieria odpowiedzi*. Ta praktyka polega na opracowywaniu technik konstruowania zapytań, nazywanych *promptami* lub *podpowiedziami*, które mają na celu optymalizację pracy systemów sztucznej inteligencji.

W ramach promptingu istnieją różne techniki formułowania promptu dla dużych modeli językowych. Wyróżnić można dwa elementy, które zazwyczaj pojawiają się w podpowiedzi: *instrukcja* z opisem zadania, oraz *przykłady* demonstrujące poprawne rozwiązanie zadania. Najważniejsze w kontekście tej pracy techniki to:

Few-shot learning Metoda wprowadzona w pracy pt. „Language Models are Few-Shot Learners” [7], zawierająca kilka przykładów zadania z poprawnymi rozwiązaniami oraz instrukcję.

Zero-shot learning Podany prompt składa się wyłącznie z instrukcji i kontekstu, bez podawania przykładów.



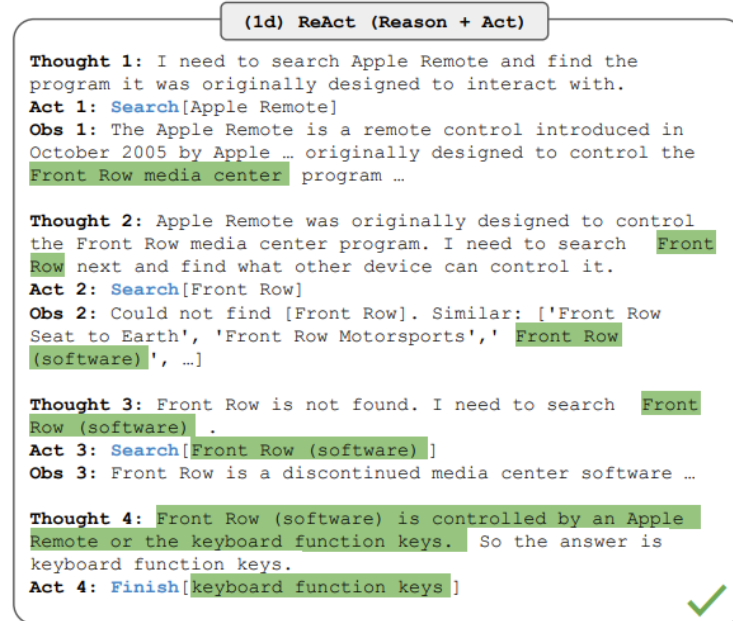
RYSUNEK 2.1: Przykład odpowiedzi modelu, w którym zastosowane zostały techniki promptingu Few-shot, Zero-shot, jak i łańcucha myśli w wersji Few-shot i Zero-shot. Na przykładzie widoczna jest poprawa zdolności modelu poprzez zastosowanie łańcucha myśli. Przykład z pracy „Large Language Models are Zero-Shot Reasoners” [20].

Łańcuch myśli (CoT, ang. *Chain of Thought*) [37] Strategia wzbogacająca przykłady o pośrednie kroki rozumowania, prowadzące do prawidłowej odpowiedzi poprzez dodanie do instrukcji polecenia w stylu „Rozwiąż to zadanie krok po kroku” [40]. Badania wskazują na istotną skuteczność tego podejścia do konstruowania promptów [37, 21]. Metoda ta odzwierciedla ludzką praktykę wizualizacji toku myślenia poprzez zapisywanie poszczególnych etapów rozwiązania problemu, zanim zostanie przedstawiony ostateczny wynik. Na rysunku 2.1 został przedstawiony przykład użycia tej strategii promptingu oraz porównanie do technik Zero-shot i Few-shot.

ReAct (ang. *Reasoning and Acting*) [42] Technika łącząca generowanie śladów rozumowania (ang. *reasoning traces*) z działaniami specyficznymi dla zadania (ang. *task-specific actions*). ReAct pozwala modelom językowym na indukcję, śledzenie i aktualizację planów działań oraz obsługę wyjątków. Umożliwia także interakcję z zewnętrznymi źródłami informacji, takimi jak bazy wiedzy, co prowadzi do bardziej wiarygodnych odpowiedzi. Badania wykazały, że ReAct może przewyższać wiele najnowocześniejszych metod w zadaniach językowych i decyzyjnych, jednocześnie poprawiając interpretowalność i wiarygodność modeli językowych [42]. Przykład wykorzystania przedstawiono na Rysunku 2.2.

Ograniczenia okna kontekstowego Istotnym aspektem wpływającym na efektywność technik promptingu jest *okno kontekstowe* (ang. *context window*), czyli maksymalna liczba tokenów, które model może przetworzyć jednocześnie. Ograniczenie to wynika z architektury modeli, przez co przy projektowaniu systemów wykorzystujących prompting należy brać pod uwagę następujące elementy:

- **Długość promptu:** Zbyt długie prompty mogą zostać obcięte, co prowadzi do utraty istotnych informacji lub przykładów.
- **Złożoność zadań:** Zadania wymagające analizy długich tekstów lub wieloetapowego rozumowania mogą być trudne do zrealizowania w ramach jednego promptu.
- **Pamięć kontekstowa:** Model „zapomina” informacje spoza bieżącego okna, co może prowadzić do niespójności w długich interakcjach.
- **Optymalizacja promptów:** Konieczność starannego doboru i kompresji informacji w promptach, aby zmieścić się w limicie okna kontekstowego.



RYSUNEK 2.2: Przykład zastosowania techniki ReAct w zadaniu odpowiadania na pytania. Widoczne są etapy rozumowania (Thought) przeplatane z konkretnymi działaniami (Act) i obserwacjami (Obs), co prowadzi do końcowej odpowiedzi. Przykład z pracy „React: Synergizing reasoning and acting in language models” [42].

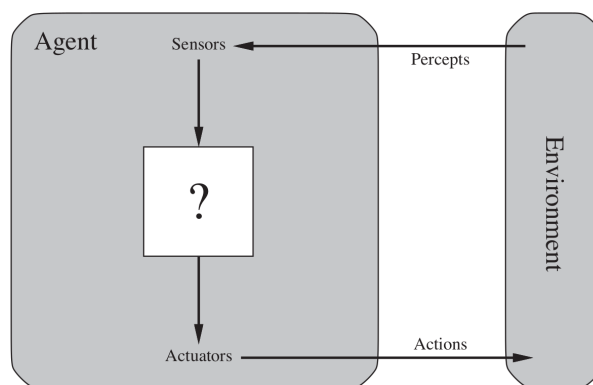
Pomimo ciągłego wzrostu wielkości okien kontekstowych w nowszych modelach, obserwowany jest problem znany jako „igła w stogu siana” (ang. *needle in the haystack*). Odnosi się on do sytuacji, gdy kluczowa informacja jest „zagubiona” wśród dużej ilości mniej istotnych danych w obrębie okna kontekstowego. Przy bardzo dużych oknach kontekstowych, model może mieć jeszcze większe trudności z wyszukaniem i wykorzystaniem najważniejszych informacji, szczególnie gdy są one rozproszone w długim tekście. Problem ten podkreśla, że samo zwiększanie rozmiaru okna kontekstowego nie jest wystarczającym rozwiązaniem i wskazuje na potrzebę rozwoju bardziej zaawansowanych technik przetwarzania i priorytetyzacji informacji w obrębie promptu [25].

2.1.2 Zdolności dużych modeli językowych w perswazji

Perswazja to sztuka przekonywania innych do przyjęcia określonego punktu widzenia lub podjęcia konkretnych działań. W kontekście sztucznej inteligencji, a szczególnie dużych modeli językowych, perswazja nabiera nowego, potencjalnie niebezpiecznego wymiaru [14]. Modele te, dzięki zdolności do prowadzenia naturalnych, długotrwałych rozmów i personalizacji treści, mogą skutecznie wpływać na opinie i zachowania użytkowników. Główne zagrożenia związane z tą technologią obejmują możliwość manipulacji odbiorcami przez dezinformację [3, 5, 8, 22].

W odpowiedzi na te zagrożenia Komisja Europejska zaproponowała zakaz wykorzystywania systemów AI, które „używają technik podprogowych wykraczających poza świadomość osoby lub celowo manipulacyjnych bądź zwodniczych, z zamiarem lub skutkiem istotnego zniekształcenia zachowania osoby lub grupy osób, poprzez znaczące osłabienie zdolności tej osoby do podejmowania świadomej decyzji, powodując tym samym, że osoba ta podejmuje decyzję, której w przeciwnym razie by nie podjęła, w sposób, który powoduje lub może spowodować, że ta osoba, inna osoba lub grupa osób poniesie znaczną szkodę” (tłumaczenia własne) [15, p. 131]. W wyniku regulacji zaistniała potrzeba opracowania metod oceny zdolności perswazyjnych systemów AI.

Badanie [13] przeprowadzone przez Anthropic analizowało zdolności perswazyjne różnych modeli językowych, w tym Claude 3 Opus. Wyniki pokazały, że wraz z rozwojem kolejnych generacji



RYСУNEK 2.3: Ilustracja interakcji agenta ze środowiskiem. Rysunek z książki [31].

modeli, ich skuteczność w przekonywaniu ludzi rośnie, osiągając poziom perswazji porównywalny z ludźmi w przypadku najlepszego modelu. Ograniczeniami badania były, m.in.: ocena zdolności perswazji w pojedynczych turach konwersacji oraz to, że ludzie oceniający perswazyjność nie byli ekspertami.

Obecnie brakuje standardowych benchmarków do pomiaru zdolności perswazji. Obiecującym podejściem mogłoby być testowanie inteligentnych agentów bazujących na dużych modelach języka w środowisku gry, w której kluczowa jest perswazja.

2.2 Inteligentny agent

2.2.1 Klasyczne definicje inteligentnych agentów

W swojej fundamentalnej pracy na temat sztucznej inteligencji, S. Russell i P. Norvig przedstawiają następującą definicję agenta: „*Agent* to cokolwiek, co można postrzegać jako byt odbierający swoje *środowisko* (ang. *environment*) poprzez *sensory* (ang. *sensors*) i oddziałujący na to środowisko za pomocą *efektorów* (ang. *actuators*)” [31], gdzie sensory to elementy, które służą do dokonywania *obserwacji* (ang. *percept*) środowiska przez agenta, a efekторы to elementy, które pozwalają agentowi wpływać na środowisko poprzez wykonywanie *akcji* (ang. *actions*). Idea ta została przedstawiona na Rysunku 2.3.

Przedstawiając to na przykładzie prostego inteligentnego odkurzacza, którego celem jest odkurzenie danego pomieszczenia: agentem jest wspomniany odkurzacz, który na podstawie swoich obserwacji, takich jak odczyt z kamer i czujników dotyku, potrafi wykonać akcje takie jak jazda w przód, skręt w lewo lub prawo, czy też zatrzymanie się za pomocą swojego efektora, którym jest napęd tego odkurzacza. W aktualnym stanie odkurzacz ten wykonuje akcje biorąc pod uwagę tylko obecnie obserwowany stan środowiska, w którym się znajduje, a co za tym idzie, nie posiada informacji czy w danym miejscu już był i może krążyć bez celu. Konieczna jest zatem *historia obserwacji* (ang. *percept sequence*), które agent mógł zaobserwować podczas swojego działania, jednak tak jak zauważają Russell i Norvig, „wybór działania agenta w danej chwili może zależeć od całej sekwencji obserwacji dokonanych do tej pory, ale nie od czegoś, czego nie zaobserwował” [31]. Dzięki temu w naszym przykładzie z odkurzaczem agent będzie znał stany, w których był i będzie odwiedzał tylko miejsca, które przybliżą go do osiągnięcia celu, jakim jest odkurzenie całego pomieszczenia.

W kontekście działania agenta kluczowe jest zrozumienie pojęć *funkcji agenta* (ang. *agent function*) i *programu agenta* (ang. *agent program*). Różnica między tymi pojęciami jest następująca: funkcja agenta to teoretyczny opis zachowania agenta, podczas gdy program agenta to praktyczna

realizacja tego zachowania w formie wykonywalnego algorytmu. Dzięki odpowiedniemu zastosowaniu przedstawionych koncepcji nasz inteligentny odkurzacz będzie posiadał logikę, która pozwoli mu zrealizować cel, na przykład: jeżeli historia obserwacji wskazuje, że miejsce do którego zmierza zostało przez niego oczyszczone, to zmieni kierunek. Aby móc zweryfikować, czy podczas działania agenta osiągany jest cel, wymagana jest *miara jakości* (ang. *performance measure*), która dokonuje oceny zachowania agenta w danym środowisku. W naszym przypadku może to być ocena jak dokładnie odkurzone jest pomieszczenie. Gdy dany agent zachowuje się w taki sposób, aby przy aktualnie posiadanej historii obserwacji maksymalizować wartość oczekiwaną miary jakości, to można stwierdzić, że mamy do czynienia z *agentem racjonalnym* (ang. *rational agent*).

Środowisko

Po przedstawieniu podstawowych koncepcji inteligentnego agenta kluczowe jest omówienie środowiska, w którym agent działa. Środowisko to wszystko co otacza agenta i z czym agent wchodzi w interakcję. Jest ono ściśle związane z funkcjonowaniem agenta, wpływając na jego percepcję, decyzje i efektywność działań, co oddziałuje na projektowanie odpowiednie rozwiązania programu agenta. Wracając do przykładu z inteligentnym odkurzaczem, środowiskiem jest pomieszczenie, które ma zostać odkurzone. W środowisku występują takie elementy jak podłoga, ściany, meble, a także możliwym jest pojawienie się czynników dynamicznych, takich jak poruszający się człowiek czy kot. Aby agent mógł działać w środowisku, muszą być zdefiniowane wspomniane wcześniej elementy, takie jak: miara jakości, środowisko, efektory i sensory (**PEAS**, ang. *Performance measure, Environment, Actuators, Sensors*).

Środowiska, w których działają agenci, mogą być niezwykle zróżnicowane, a każde z nich może charakteryzować się specyficznym poziomem złożoności. Dlatego, aby lepiej zrozumieć i scharakteryzować środowisko, w którym działa agent, niezbędna jest analiza jego kluczowych właściwości. Russell i Norvig [31] definiują siedem głównych właściwości środowiska, które są kluczowe dla zrozumienia poziomu wyzwań, przed którymi staje agent:

- **Całkowicie obserwowalne i częściowo obserwowalne** (ang. *fully observable vs. partially observable*) – określa, czy agent ma dostęp do pełnego stanu środowiska w danym momencie, czy tylko częściowy dostęp. Całkowicie obserwowalne środowiska definiujemy, gdy sensory agenta posiadają pełen dostęp do stanu, czyli sensory posiadają wszystkie istotne informacje do podjęcia akcji, w konsekwencji czego agent nie musi pamiętać poprzednich stanów. Przykład: gra w szachy, gdzie plansza i pozycja figur jest zawsze widoczna dla agenta. Dla częściowo obserwowalnych środowisk agent nie posiada pełnego stanu wiedzy i musi podejmować decyzje na podstawie niepełnych danych. Agent musi wtedy radzić sobie z niekompletnością informacji i pamiętać poprzednie stany środowiska. Przykład: gra w pokera, gdzie agent zna tylko swoje karty, a nie zna kart przeciwnika.
- **Jednoagentowe i wieloagentowe** (ang. *single-agent vs. multiagent*) – definiuje, czy w środowisku działa tylko jeden agent, czy też jest ich wielu. Środowiska wieloagentowe mogą mieć charakter kompetytywny (gdy rywalizujemy z innymi agentami) lub kooperatywny (gdy współpracujemy z innymi agentami). Dla przykładu: przy umieszczeniu kilku inteligentnych odkurzaczy w jednym pomieszczeniu, kiedy celem będzie najszybsze posprzątanie pomieszczenia – agenci będą kooperować. Z kolei w sytuacji, gdy za cel postawi się posprzątanie jak największej powierzchni przez danego agenta – będą oni rywalizować. Agenci dla środowisk wieloagentowych często wymagają dodatkowych usprawnień, takich jak wprowadzenie moż-

liwości komunikacji między kooperującymi agentami, lub dodanie elementu losowości w celu zmylenia przeciwnika dla agentów kompetytywnych.

- **Deterministyczne** i **niedeterministyczne** (ang. *deterministic vs. nondeterministic*) – wskazuje, czy następny stan środowiska jest całkowicie określony przez stan obecny i akcję agenta, czy też następny stan jest zależny od elementu losowości lub niepewności. W środowisku deterministycznym, znając jego aktualny stan i akcję agenta wiemy, w jakim stanie jest środowisko po wykonaniu akcji, w przeciwnym wypadku mówimy o środowisku niedeterministycznym. Przykładowo w deterministycznym środowisku inteligentnego odkurzacza, agent wykonujący akcje poruszać się będzie zgodnie z ruchem, jaki wykonały koła. W przeciwnym wypadku, gdyby środowisko było niedeterministyczne, koła agenta mogłyby wpaść w poślizg, powodując nieprzewidzianą zmianę pozycji. Środowisko niedeterministyczne jest bardziej złożone dla agenta, gdyż musi on brać pod uwagę niepewność w procesie wyboru akcji.
- **Epizodyczne** i **sekwencyjne** (ang. *episodic vs. sequential*) – określa, czy doświadczenie agenta dzieli się na niezależne epizody, czyli akcje są wykonywane w sposób niezależny od siebie, czy też każda akcja wpływa na przyszłe stany i decyzje. Przykładem środowiska epizodycznego może być stwierdzanie co epizod czy dane zdjęcie jest sztucznie wygenerowane. Jako jeden z przykładów środowiska sekwencyjnego można wymienić szachy, gdzie każdy wykonany ruch wpływa na następny. Epizodyczne środowiska są znacznie łatwiejsze dla agenta, gdyż nie musi on brać pod uwagę przyszłych konsekwencji swoich działań.
- **Statyczne** i **dynamiczne** (ang. *static vs. dynamic*) – definiuje, czy środowisko może się zmieniać podczas procesu podejmowania decyzji przez agenta, czy pozostaje niezmiennie. W przykładzie z inteligentnym odkurzaczem, gdy w pomieszczeniu będzie znajdował się chodzący kot, którego trzeba ominąć, to środowisko byłoby dynamiczne, w przeciwnym wypadku, gdy w pomieszczeniu jedynym ruchomym bytem byłby odkurzacz, środowisko należałoby zaliczyć jako statyczne. Dynamiczne środowiska są trudniejsze dla agentów, gdyż agent musi być zoptymalizowany pod względem czasu, jakiego potrzebuje na podjęcie decyzji, oraz musi ciągle monitorować i reagować na zmiany.
- **Dyskretnie** i **ciągłe** (ang. *discrete vs. continuous*) – określa, czy stany środowiska zmieniają się w sposób dyskretny, czy ciągły. Środowiska dyskretnie charakteryzują się skończoną liczbą stanów i akcji, w przeciwieństwie do środowisk ciągłych. Środowiskiem dyskretnym mogą być gry planszowe, takie jak szachy, gdzie każda pozycja jest dyskretnie określona. W środowisku ciągłym precyzyjne położenie odkurzacza w pokoju będzie definiowane przez parę liczb rzeczywistych. Środowiska ciągłe wymagają większej złożoności obliczeniowej i zazwyczaj potrzebne są metody aproksymacji funkcji, aby radzić sobie z nieskończoną przestrzenią stanów.
- **Znane** i **nieznane** (ang. *known vs. unknown*) – wskazuje, czy agent zna zasady funkcjonowania środowiska (tj. model przejść między stanami i funkcję nagrody), czy musi je odkrywać w trakcie działania na przykład: gra w kółko i krzyżyk. Agent w środowisku nieznanym wymaga strategii eksploracji w celu dokładniejszego poznania środowiska, czego przykładem może być inteligentny odkurzacz w pomieszczeniu, którego wcześniej nie widział. W środowiskach nieznanych istnieje potrzeba mechanizmów ciągłego uczenia się środowiska.

Zauważyć więc można, że dobra znajomość środowiska jest kluczowa podczas projektowania agentów, którzy w tym środowisku będą wykonywać działania. Najtrudniejszym typem środowiska dla

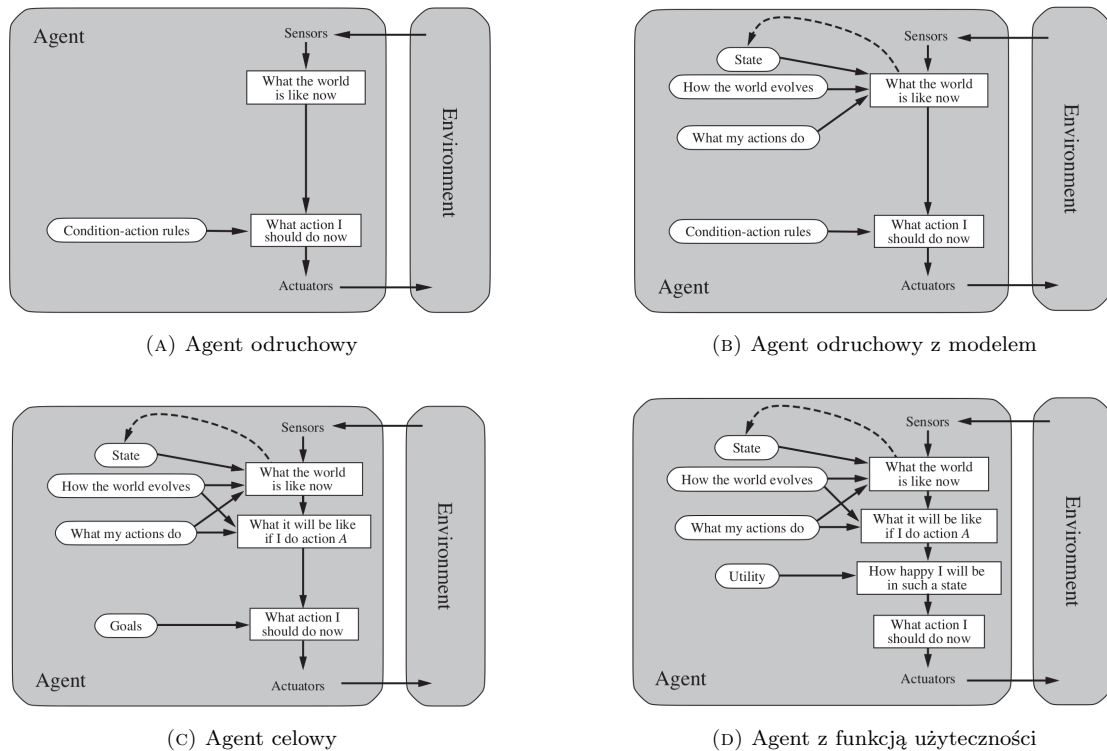
danego agenta byłoby częściowo obserwowalne, wieloagentowe, niedeterministyczne, sekwencyjne, dynamiczne, ciągle i nieznane środowisko. Przykładem tak trudnego środowiska może być wspomniany inteligentny odkurzacz (*sekwencyjne*), którego sensory nie widzą całego pomieszczenia (*częściowo obserwowalne*), w pomieszczeniu znajdują się inne inteligentne odkurzacze (*wieloagentowe*), które trzeba omijać (*dynamiczne*), koła narażone są na wpadnięcie w poślizg (*niedeterministyczne*), cechy środowiska takie jak pozycja są wyrażone w liczbach rzeczywistych (*ciągle*) oraz agent znajduje się w tym środowisku po raz pierwszy i nie zna rządzących nim reguł (*nieznane*).

Agent

Przedstawione do tej pory terminy dotyczyły specyfiki środowiska i tego, co dany agent może spoznać za pomocą sensorów, a następnie jak może za pomocą efektorów wpływać na to środowisko poprzez akcje. Główną rolą sztucznej inteligencji w projektowaniu agentów jest stworzenie wspomnianego wcześniej *programu agenta*, który określa, jak agent będzie się zachowywał: jaką akcję wybierze po dokonaniu danej obserwacji. Środowisko dostarcza agentowi tylko obecną obserwację, a gdy do podjęcia decyzji wymagana jest historia obserwacji, to zadaniem agenta jest utrzymywanie historii obserwacji. Russel i Norvig [31] rozróżniają 4 podstawowe typy programów agentów, w kolejności od najprostszych do najbardziej ogólnych:

- *Agent odruchowy* (ang. *simple reflex agent*) – najprostszy w implementacji i swojej logice. Wybiera akcje na podstawie systemu regułowego, który można przedstawić jako: **jeżeli** obecna obserwacja **to** wykonaj akcje. Nie posiada historii swoich obserwacji. Potrafi być racjonalny w środowiskach całkowicie obserwowalnych. Schemat działania został przedstawiony na Rysunku 2.4a.
- *Agent odruchowy z modelem* (ang. *model-based reflex agent*) – wzbogaca model odruchowy poprzez wewnętrzny stan, w którym przechowywana jest historia obserwacji o tym jak zmienia się świat oraz jakie efekty przynoszą akcje. Wewnętrzny stan musi być zawsze uaktualniany przez agenta po dokonaniu nowej obserwacji. Dzięki utrzymywaniu wiedzy o poprzednich obserwacjach ten rodzaj agenta potrafi być racjonalny również w częściowo obserwowalnych środowiskach. Schemat działania został przedstawiony na Rysunku 2.4b.
- *Agent celowy* (ang. *goal-based agent*) – agent ten różni się od poprzedników tym, że zamiast reguł, ma zdefiniowany cel, który musi osiągnąć, i potrafi wskazać konsekwencje swoich akcji. Agent zazwyczaj, aby osiągnąć swój cel, musi wykonywać przeszukiwanie lub planowanie. Schemat działania został przedstawiony na Rysunku 2.4c.
- *Agent z funkcją użyteczności* (ang. *utility-based agent*) – zamiast wyznaczonego celu, agent oblicza dla każdej akcji na podstawie *oczekiwanej funkcji użyteczności*, jak bardzo stan, w którym się znajdzie po wykonaniu akcji, jest przez niego pożądanym. Funkcja użyteczności stara się przybliżyć wspomnianą na początku rozdziału miarę jakości agenta, a gdy jest z nią zgodną, to agent będzie zachowywał się w sposób racjonalny. Ten typ programu agenta jest szczególnie użyteczny w dwóch przypadkach: gdy agent ma wiele celów oraz gdy cele są osiąganym z pewnym prawdopodobieństwem. Pozwala to na osiągnięcie kompromisów pomiędzy różnymi celami, a także na efektywne działanie w warunkach niepewności co do skutków podejmowanych akcji.

Każdy z przytoczonych typów programów agenta przedstawia, w jaki sposób wybierana jest akcja po otrzymaniu obserwacji. Jednakże, w większości rzeczywistych zastosowań, środowiska są zbyt złożone lub dynamiczne, aby agent mógł działać efektywnie z zaprogramowanymi regułami



RYСУNEK 2.4: Diagramy przedstawiające działanie agentów. Prostokątem oznaczone są wewnętrzne stany agenta podczas procesu decyzji. Owale służą do reprezentacji informacji użytej do podjęcia decyzji. Ilustracja z książki [31].

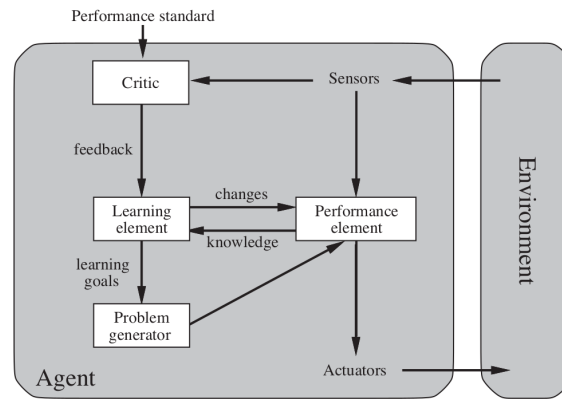
lub statyczną funkcją użyteczności. W takich przypadkach kluczową rolę odgrywa zdolność agenta do uczenia się i adaptacji. Wszystkie wymienione typy można przekształcić w *agenta uczącego się* (ang. *learning agent*) i na bieżąco udoskonalać podejmowane decyzje poprzez interakcje ze środowiskiem. Agent uczący się działa zgodnie z Rysunkiem 2.5 i składa się z 4 elementów:

1. *Element wykonawczy* (ang. *performance element*) – odpowiedzialny za wybór akcji. Wcześniej nazywany programem agenta.
2. *Element uczący* (ang. *learning element*) – odpowiedzialny za wprowadzanie zmian w działaniu i ulepszanie agenta.
3. *Krytyk* (ang. *critic*) – ocenia jak agent sobie radzi oraz ustala, jak element wykonawczy powinien zostać zmodyfikowany, aby lepiej sobie radzić w przyszłości. Ocena skuteczności agenta jest tworzona zgodnie z ustaloną *miarą jakości* (ang. *performance standard*) [31].
4. *Element eksploracyjny* (ang. *problem generator*) – odpowiedzialny za eksplorację nowych, wcześniej nie doświadczanych przez agenta, stanów, lub dostarczanie mu nowych informacji.

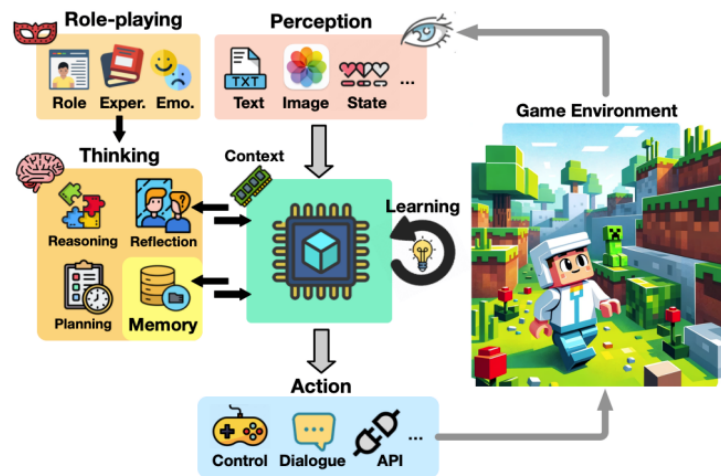
Dzięki przekształceniu do agenta uczącego się, agenci odruchowi są w stanie udoskonalać swoje reguły decyzyjne, agenci celowi mają możliwość optymalizacji strategii osiągania celów, natomiast agenci z funkcją użyteczności mogą dostosowywać tę funkcję na podstawie zdobytych doświadczeń.

2.2.2 Agent bazujący na dużym modelu językowym

Ze względu na duże postępy związane z rozwojem dużych modeli języka i ich zdolności w rozumieniu języka naturalnego (*NLU*, ang. *Natural Language Understanding*), w ostatnim czasie zaczęły powstawać prace na temat wykorzystania wspomnianych modeli do projektowania inteligentnych



RYSUNEK 2.5: Diagram agenta uczącego się. Rysunek z książki [31].



RYSUNEK 2.6: Diagram zunifikowanej architektury agentów bazujących na dużym modelu językowym. Rysunek z pracy przeglądowej [16].

agentów. *Agenta bazującego na dużym modelu językowym (LLMA, ang. LLM-based agent)* można zdefiniować jako byt, który wykorzystuje LLM jak swój główny komponent, aby realizować postawione przed nim zadania. Zastosowania tych agentów wykraczają poza środowiska gier, dlatego powstał termin, który odnosi się wyłącznie do agentów działających w środowisku gier, i nosi nazwę *agenta gier bazującego na dużym modelu języka (LLMGA, ang. LLM-based game agent)*. Zaletą wykorzystania LLM jest możliwość wiarygodnej interakcji z człowiekiem poprzez język naturalny oraz umiejętność poruszania się w środowiskach, w których postęp w grze przekazywany jest w sposób tekstowy. LLMGA mogą również zostać wykorzystane w środowiskach wymagających rozumienia graficznych obrazów. Wykorzystuje się wtedy multimodalne duże modela języka (*MLLM, ang. Multimodal Large Language Model*). W pracy przeglądowej skupiającej się na LLMGA [16] autorzy przedstawiają widoczną na Rysunku 2.6 zunifikowaną architekturę dla agentów bazujących na dużym modelu języka, gdzie wymieniono 6 głównych komponentów składających się na architekturę agenta:

1. Obserwacje (ang. *perception*) – podobnie jak w klasycznych agentach, komponent odpowiedzialny za wykonywanie obserwacji świata. W przeciwieństwie do klasycznych agentów, poza obserwacją stanu środowiska jest w stanie obserwować i przetwarzać obserwacje, w których informacja zawarta jest w formie tekstu lub w formie obrazka, gdy użyty model jest multimodalny.

2. Pamięć (ang. *memory*) – służy jako repozytorium poprzednich obserwacji, myśli i akcji, z których komponent ten wyszukuje potrzebne informacje dla procesu decyzyjnego czy tworzenia strategii. Pamięć można podzielić na 3 różne rodzaje:

- Pamięć epizodyczna (ang. *episodic memory*) – służy jako pamięć trwała do przechowywania historii epizodów. Mogą to być wszystkie historyczne obserwacje lub wydarzenia z poprzednich gier, które pochodzą z obserwacji agenta.
- Pamięć semantyczna (ang. *semantic memory*) – wykorzystywana do przechowywania „przemyśleń” agenta, czyli to jak rozumie, to kim jest, gdzie jest i co ma wykonać. Pamięć ta pochodzi z modułu *rozumowania* opisanego poniżej.
- Pamięć proceduralna (ang. *procedural memory*) – przechowuje wyselekcjonowane umiejętności (ang. *curated skills*) jako kod lub plany.

Wraz z działaniem agenta, wspomnienia potrafią się nagromadzić i zakłócić proces decyzyjny, dlatego, w celu optymalizacji, często stosowane są techniki wyszukiwania informacji tak, by podać agentowi tylko istotne w danej chwili informacje.

3. Myślenie (ang. *Thinking*) – komponent odpowiedzialny za analizę i integrację informacji. W kontekście LLMGA możemy wyróżnić dwie główne metody myślenia wykorzystywane w podejmowaniu decyzji:

- Rozumowanie (ang. *reasoning*) – rozumowanie to proces, który rozpoczyna się od obserwacji, dowodów faktycznych i wcześniejszych przemyśleń. W ramach tego procesu wprowadzane są szczegółowe wnioski z ogólnych zasad (dedukcja), wnioskowane są ogólne zasady na podstawie analizy konkretnych przypadków (indukcja), oraz formułowane są najbardziej prawdopodobne wyjaśnienia (abdukcja). Przy rozumowaniu korzysta się z różnych technik, takich jak *ReAct* (ang. *Reasoning + Acting*) [42] czy *CoT* (ang. *Chain of Thought*) [37] opisanych w tej pracy w sekcji 2.1.1. Szczególnym rodzajem rozumowania jest *refleksja* (ang. *reflection*), która pozwala agentom analizować własne działania i procesy myślowe, prowadząc do doskonalenia strategii i zachowań.
- Planowanie (ang. *planning*) – planowanie obejmuje dekompozycję złożonego zadania na prostsze podzadania. Planowanie może być *ukierunkowane na cel* (ang. *goal-conditioned planning*), wtedy jest zazwyczaj wykorzystywane do wykonywania zadań złożonych i długoterminowych. Pozwala na rekurencyjne przekształcanie celu w mniejsze podcele. Planowanie może również być *bez ustalonego celu* (ang. *goal-free planning*), które może wystąpić w grach świata otwartego. Wtedy generowanie planów pomaga agentowi uniknąć przytłoczenia licznymi możliwymi akcjami, co pozwala uniknąć niespójnego wykonywania akcji.

Proces rozumowania jest najważniejszym komponentem w LLMGA, który umożliwia agentom analizę sytuacji, formułowanie strategii i podejmowanie decyzji w sposób zbliżony do ludzkiego, co przyczynia się do bardziej realistycznych i adaptacyjnych zachowań w środowisku gry.

4. Granie ról (ang. *Role-playing*) – jest to kluczowy element w tworzeniu immersyjnych doświadczeń w grach oraz w budowaniu realistycznych postaci niezależnych (*NPC*, *Non Playable Character*). Odgrywanie ról umożliwia agentom przyjmowanie cech i historii różnorodnych postaci w grze, generując wiarygodne konwersacje i zachowania.

5. Wykonywanie akcji (ang. *Action*) – moduł akcji w *LLMGA* pełni kluczową rolę w przekładaniu decyzji podjętych przez agenta na konkretne działania w środowisku gry. Najprostszym przypadkiem, w którym występuje przekładanie myśli na akcje, może być typ *gier bez kontroli manipulacyjnej* (ang. *games without manipulative control*), gdzie wymaga się, aby LLM dokonał wyboru ze zbioru danych akcji lub wygenerował akcję słowo po słowie. *Gry z kontrolą manipulacyjną* (ang. *games with manipulative control*) wymagają akcji niskopoziomowych, takich jak ruch myszką czy wykonanie wciśnięcia kombinacji klawiszy na klawiaturze. LLM generują akcje wysokopoziomowe, które trzeba przetłumaczyć na sekwencje akcji niskopoziomowych za pomocą heurystyk.
6. Uczenie się (ang. *Learning*) – umożliwia uczenie i poprawę umiejętności agenta poprzez interakcje ze środowiskiem gry, podobnie jak agenci opisani w rozdziale 2.2.1. Jednak ze względu na specyfikę uczenia dużych modeli językowych, stosuje się różne techniki ich uczenia, takie jak uczenie w kontekście, nadzorowane dostrajanie [41] i uczenie ze wzmocnieniem.

Przestawiona powyżej zunifikowana architektura agenta bazującego na dużych modelach języka nie jest jeszcze standardem, lecz jedną z zauważalnie najczęściej występujących architektur w pracach o tematyce *LLMGA*. Pomimo dużych możliwości w tworzeniu realistycznych i podobnych w zachowaniu do ludzi agentów bazujących na dużych modelach języka, badacze wskazują na ograniczenia [38], z którymi muszą się mierzyć w ich tworzeniu:

- Ograniczone okno kontekstowe – ogranicza zarówno podawanie historycznych obserwacji, jak i obserwacji z poprzednich gier. Nawet jeżeli dany model języka posiada duże okno kontekstowe, to wykazano, że LLM mają problemy ze znalezieniem informacji, która będzie przydatna, spośród dużej ilości informacji zawartej w kontekście [18].
- Wyzwania dotyczące długoterminowego planowania i tworzenia podzadań – *LLMGA* mają problemy z planowaniem na podstawie długiej historii i nieefektywnie eksplorują przestrzeń rozwiązań. Wykazują również ograniczoną adaptację planów w obliczu nieoczekiwanych błędów.
- Niezawodność interfejsów opartych o język naturalny – istnieje ryzyko błędów formatowania w wyjściu modelu oraz sporadycznych „buntowniczych” zachowań LLM, czego przykładem może być odmowa wykonania instrukcji.

Prace wykorzystujące agentów bazujących na dużych modelach języka

Autorzy pracy przeglądowej na temat *LLMGA* [16] wyróżnili 6 kategorii gier, w których obecnie wykorzystuje się wspomnianych agentów: komunikacyjne, rywalizacyjne, kooperacyjne, przygodowe, symulacyjne oraz gry polegające na tworzeniu i eksploracji. Każda z tych kategorii gier stawia różne wyzwania przed agentem i wymaga innego zestawu umiejętności. W celu lepszego zrozumienia idei oraz możliwego wykorzystania agentów, przedstawione zostaną przykłady istniejących implementacji *LLMGA* w grach.

Generative Agents: Interactive Simulacra of Human Behavior [28] Stworzona została gra symulacyjna wykorzystująca interaktywne środowisko z otwartym światem inspirowanym grą „The Sims”. Jest to jedna z pierwszych prac na temat *LLMGA*, zainspirowała ona autora niniejszej pracy magisterskiej do podjęcia tematu. Środowisko to było wieloagentowe i znajdowało się w nim jednocześnie 25 różnych agentów, z których każdy miał nadaną zarówno własną osobowość, jak i relacje z innymi agentami. Praca wprowadza pojęcie *generatywnych agentów* zdolnych do

symulowania wiarygodnego ludzkiego zachowania. Agenci potrafią wykonywać codzienne czynności, formować opinie, wchodzić w interakcje z innymi agentami oraz tworzyć plany na przyszłość. Architektura agenta została stworzona w taki sposób, aby przechowywać doświadczenia agenta w języku naturalnym, syntetyzować wspomnienia w refleksje oraz dynamicznie pozyskiwać informacje do planowania zachowań. W ramach ewaluacji agencji wykazali wiarygodne zachowania indywidualne i społeczne, czego przykładem może być udana organizacja imprezy walentynkowej. W tym przykładzie tylko jeden z agentów dostał za zadanie organizację i rozpropagowanie pośród innych agentów wiedzy o wspomnianej imprezie. Pomimo wielu możliwości niepowodzenia (agent może zignorować prośbę, zapomnieć przekazać informacje innym lub zapomnieć się zjawić na imprezie), w przyjęciu walentynkowym wzięło udział 5 agentów, z czego dwóch było umówionych na randkę ze sobą. Badania autorów wykazały, że kluczowe komponenty architektury agenta (obserwacja, planowanie i refleksja) przyczyniają się do zwiększenia wiarygodności zachowań agenta.

Cradle: Empowering Foundation Agents Towards General Computer Control [33, 34]

Autorzy pracy przedstawiają ideę *Ogólnej Kontroli Komputera (GCC, ang. General Computer Control)*, czyli ujednoliconego interfejsu dla agentów AI, wykorzystującego zrzuty ekranu jako dane wejściowe oraz akcje klawiatury i myszy jako dane wyjściowe. GCC ma na celu umożliwienie stworzenia uniwersalnych agentów AI zdolnych do wykonywania zadań w każdym środowisku cyfrowym oraz przyczynić się do rozwoju *ogólnej sztucznej inteligencji (AGI, ang. Artificial General Intelligence)*. W ramach pracy stworzono framework *Cradle*, oparty na dużych modelach językowych, który realizuje założenia *GCC*. Cradle składa się z sześciu kluczowych modułów: zbieranie informacji (ang. *Information Gathering*), autorefleksja (ang. *Self-Reflection*), wnioskowanie o zadaniach (ang. *Task Inference*), dobór umiejętności (ang. *Skill Curation*), planowanie działań (ang. *Action Planning*) oraz pamięć (ang. *Memory*). Pomimo tego, że miejscami użyta została inna nomenklatura, architektura frameworku Cradle jest bardzo zbliżona do opisanej w tej pracy zunifikowanej architektury dla agentów bazujących na dużych modelach językowych. Framework wykazał wysoką skuteczność w realizacji zadań w złożonych grach wideo i popularnych aplikacjach, bez korzystania z dedykowanych API. Osiągnięcia Cradle obejmują interakcje z grami (np. *Red Dead Redemption 2*, *Cities: Skylines*) oraz aplikacjami użytkowymi (np. *Chrome*, *Outlook*). Praca ta jest stale poszerzana o wsparcie nowe gier czy narzędzi.

Hoodwinked: Deception and Cooperation in a Text-Based Game for Language Models [26]

Praca ta przedstawia grę tekstową o nazwie „Hoodwinked”, która jest inspirowana grami typu *Mafia* i *Among Us*. Celem pracy było badanie zdolności modeli językowych do oszukiwania i wykrywania kłamstw. Scenariusz gry wyglądał następująco: gracze są uwięzieni w domu i muszą znaleźć klucz aby uciec, przy czym jeden z graczy ma za zadanie zabić innych. Po każdym morderstwie, gra ze stanu wykonywania czynności przechodzi do stanu, w którym gracze prowadzą dyskusję w języku naturalnym i głosują, kogo wykluczyć z gry. Zaproponowane środowisko było wieloagentowe, a agent w nim działający mógł być sterowany przez człowieka albo duży model językowy. Przeprowadzono eksperymenty z agentami sterowanymi przez modele GPT-3, GPT-3.5 i GPT-4. Bardziej zaawansowane duże modele językowe okazały się skuteczniejszymi zabójcami, przewyższając mniejsze modele w 18 z 24 porównań parami. Autorzy stawiają tezę, że lepsze wyniki są powodowane przez lepszą umiejętność perswazji podczas prowadzonych dyskusji. Krytycy tej pracy wskazują na prostotę oraz poleganie na szczęściu w fazie rozgrywki, w której trzeba znaleźć ukryty klucz. W szczęśliwym scenariuszu niewinny gracz może uciec z domu wykonując zaledwie 3 akcje.

Rozdział 3

Środowisko

Proponowane w tej pracy środowisko jest inspirowane grą Among Us [17]. Among Us jest odmianą wieloosobowej gry typu Mafia [11], w której to poinformowana mniejszość rywalizuje z niepoinformowaną większością. Gra wymaga i rozwija umiejętności obserwacji i analizy zachowań, wyciągania wniosków, przekonywania. Wybór tego rodzaju gier jest podyktowany chęcią zbadania możliwości dużych modeli języka w planowaniu i wykonywaniu akcji, okłamywaniu i przekonywaniu do nieprawdy. Projekt dostępny jest na platformie GitHub pod adresem <https://github.com/MarcinKorcz101/mk-ai-agents>.

Gracze dzielą się na *załogantów* (ang. *crewmates*) oraz *oszustów* (ang. *impostors*). Załoganci stanowią większość załogi, a ich celem jest wykonanie wszystkich przydzielonych im zadań i identyfikacja oszustów. Jeden lub więcej graczy są oszustami, których celem jest eliminacja załogantów, pozostając przy tym niewykrytym. Przebieg gry jest następujący:

1. Gra rozpoczyna się od przydzielenia ról graczom.
2. Załoganci poruszają się po mapie, wykonując przydzielone im zadania. Zadania do wykonania znajdują się w różnych lokalizacjach.
3. Oszuści starają się wtopić w tłum i skrycie eliminują załogantów.
4. Gdy zostanie znalezione ciało, gracze dyskutują i głosują, kogo chcą usunąć z gry.

Gra kończy się po spełnieniu przez załogantów lub oszustów warunków zwycięstwa. Załoganci wygrywają, gdy wszyscy załoganci wykonają przydzielone im zadania, lub zidentyfikują i usuną wszystkich oszustów poprzez głosowanie. Oszuści wygrywają, gdy liczba załogantów zrówna się z liczbą oszustów. Podczas normalnej rozgrywki komunikacja między graczami jest niemożliwa. Dyskusja jest dozwolona tylko podczas fazy głosowania.

Miarą jakości gracza jest zwycięstwo lub przegrana. Gracz może wykonywać akcje takie jak: przemieszczenie się do sąsiedniego pomieszczenia, wykonanie zadania, zabicie gracza, zwołanie graczy do dyskusji, gdy odkryje martwego gracza, czy czekanie w pomieszczeniu, w którym obecnie się znajduje. Gracz może zaobserwować tylko to, co stało się w pomieszczeniu w którym przebywał lub do którego wszedł. Wymienione tutaj elementy, wraz z przedstawionym opisem środowiska, definiują nam PEAS przedstawione w rozdziale 2.2.1.

Pierwotnie niniejsza praca nie miała na celu tworzenia tego środowiska od zera, lecz wykorzystanie gry Among-sus (sic) [23], która jest rekreacją mechanik Among Us, ale w trybie gry tekstowej. Jednakże, po testach oraz próbach ulepszenia tego systemu, zaistniała potrzeba stworzenia nowego środowiska, które byłoby stworzone z myślą o wykorzystaniu LLMGA oraz przeprowadzeniu eksperymentów. Istotnymi czynnikami ograniczającymi użycie „Among-sus” były:

1. Asynchroniczność gry – była to gra czasu rzeczywistego, stworzona tak, aby użytkownik, który szybciej wpisze poprawnie całe zdanie, wykonał czynność. Wprowadzało to możliwość ukończenia gry w parę sekund przez błyskawiczne wykonywanie zadań przez załogantów. W trakcie eksperymentów z LLMGA w czasie, w którym trwała inferencja dużego modelu językowego, wystąpiły zdarzenia, których agent nie uwzględniał w swojej decyzji.
2. Brak przechowywania historii rozgrywki – niemożność zapisu rozgrywki po jej ukończeniu utrudnia prowadzenie eksperymentów.
3. Jakość istniejącego kodu – kod nie był pisany z myślą o dalszym utrzymaniu, przez co wprowadzanie zmian było trudne.

W odpowiedzi na te ograniczenia stworzono nowe środowisko inspirowane grą Among Us, które wprowadza trwałość rozgrywki i przechowywanie historii rozgrywki. Dodatkowo zmieniono język programowania z C na Python oraz zastosowano programowanie obiektowe ułatwiające dalsze utrzymanie środowiska.

3.1 Właściwości środowiska

Utworzone środowisko można opisać następującymi właściwościami:

- Częściowo obserwowalne – agent w danym stanie obserwuje stan środowiska dla pomieszczenia, w którym się znajduje, lub do pomieszczenia, do którego właśnie wszedł. Dodatkowo agenci nie znają ról innych graczy.
- Wieloagentowe – w środowisku występuje wielu agentów. Środowisko to ma charakter zarówno kooperatywny, jak i kompetytywny, ponieważ załoganci muszą współpracować ze sobą w celu znalezienia oszusta i jednocześnie rywalizują oni z oszustami.
- Niedeterministyczne – załogant może zginąć w trakcie wykonywania czynności, co wprowadza element niepewności.
- Sekwencyjne – każda z wykonanych akcji wpływa na przyszłe stany i decyzje.
- Dynamiczne – środowisko może się zmienić w trakcie gry, czego przykładem może być pojawienie się ciała w pomieszczeniu lub załoganta, którego oszust może wyeliminować.
- Dyskretne – brak elementów ciągłych w stanach środowiska.
- Znane – zasady są ściśle ustalone. Każdy z agentów zna zasady gry i rozkład pomieszczeń.

3.2 Implementacja

Implementacja środowiska gry została zrealizowana z wykorzystaniem najnowszych technologii i praktyk programistycznych, co zapewniło wysoką jakość i utrzymywalność kodu. Poniżej przedstawiono kluczowe aspekty techniczne projektu:

1. **Język programowania** – środowisko zostało napisane w Pythonie 3.11, co pozwoliło na wykorzystanie najnowszych funkcji języka, w tym usprawnień w zakresie wydajności i obsługi błędów.

2. **Zarządzanie zależnościami** – do zarządzania zależnościami projektu wykorzystano narzędzie Poetry, które umożliwia proste i precyzyjne kontrolowanie wersji bibliotek, izolację środowiska projektu oraz łatwe odtworzenie środowiska deweloperskiego.
3. **Type hints** – w kodzie konsekwentnie stosowano wskazówki typów (type hints), zgodnie z najnowszymi standardami PEP 484 i PEP 585. Praktyka ta znacząco poprawiła czytelność kodu, ułatwiła wykrywanie potencjalnych błędów na etapie rozwoju oraz umożliwiła lepsze wsparcie ze strony narzędzi IDE.
4. **Statyczna analiza kodu** – w procesie rozwoju regularnie stosowano narzędzie Mypy, które służy do statycznej analizy kodu.

Powyższe praktyki i narzędzia przyczyniły się do stworzenia łatwego w utrzymaniu kodu środowiska gry, co jest szczególnie istotne w kontekście projektu akademickiego i potencjalnego dalszego rozwoju. Nie wszystkie mechaniki zostały odtworzone z gry, na której to środowisko było wzorowane. W stworzonej implementacji nie zostały wprowadzone mechaniki takie jak używanie wentylacji przez oszustów w celu szybkiego przemieszczania się, możliwość dokonania przez oszustów sabotażu, brak specjalnych pomieszczeń, w których mamy dostęp do kamer na pokładzie.

3.3 Przystosowanie do eksperymentów

Środowisko zostało dostosowane do przeprowadzania eksperymentów z inteligentnymi agentami bazującymi na dużych modelach językowych (LLMGA).

1. **Zapis przebiegu rozgrywki** – zaimplementowano mechanizm zapisu przebiegu każdej rozgrywki, co umożliwia dokładną analizę zachowań agentów i przebiegu gry. Zapis obejmuje akcje podejmowane przez agentów oraz interakcje między agentami.
2. **Mrożenie fazy akcji** – wprowadzono możliwość „mrożenia” poszczególnych faz akcji w grze. Umożliwia to przetestowanie różnych zdolności agentów, ponieważ faza rozgrywki wymaga umiejętności planowania i wykonywania akcji. Natomiast faza dyskusji skupia się na elementach dotyczących perswazji.
3. **Elastyczność konfiguracji** – środowisko umożliwia proste dostosowywanie parametrów gry, takich jak liczba graczy, rozkład ról czy specyfika i liczba zadań, co pozwala na przeprowadzanie szerokiego spektrum eksperymentów.
4. **Elastyczność w konfiguracji LLMGA** – środowisko umożliwia integrowanie różnych dużych modeli językowych, na których bazować będą inteligentni agenci.

Zastosowanie wymienionych elementów pozwala na efektywne prowadzenie eksperymentów. Stworzone środowisko stanowi nową platformę do badań nad zachowaniem inteligentnych agentów bazujących na dużych modelach językowych oraz rozszerza środowiska wymienione w pracy przeglądowej o LLMGA [16] o nowe środowiska gier opartych na komunikacji.

Rozdział 4

Agent

W niniejszym rozdziale przedstawiona zostanie implementacja agentów opartych na dużych modelach językowych (LLMGA) w środowisku gry wzorowanej na Among Us. Omówione zostaną kluczowe decyzje projektowe, struktura kodu oraz wybór konkretnych rozwiązań wraz z uzasadnieniem.

4.1 Zadania agenta

W implementacji gry typu Among Us można wyróżnić 3 różne fazy gry. Każda z faz wymaga od inteligentnego agenta innego rodzaju działania:

1. **Faza akcji** – podejmowanie decyzji o ruchach i działaniach w środowisku gry.
2. **Faza dyskusji** – uczestniczenie w dyskusji z innymi graczami, dzielenie się obserwacjami i podejrzeniami. Agent będący oszustem będzie próbował oszukiwać i zwodzić innych graczy, natomiast agent będący załogantem będzie starał się odkryć, kto jest oszustem.
3. **Faza głosowania** – podejmowanie decyzji o tym, na kogo głosować lub czy wstrzymać się od głosu.

Każde z tych zadań wymaga od agenta analizy bieżącej sytuacji, historii gry oraz strategicznego myślenia, uwzględniającego jego rolę (załogant lub oszust).

4.2 Implementacja

Z racji na wyróżnione 3 fazy gry, zdecydowano się na stworzenie 3 osobnych agentów, z których każdy jest odpowiedzialny za działania w konkretnej fazie gry. W implementacji wykorzystano framework Langchain [1], który umożliwia prostą integrację dużych modeli językowych wraz z technikami promptingu, co pozwala na efektywne wykorzystanie możliwości LLM w kontekście gry. Dodatkowo, w celu rygorystycznej kontroli typów oraz walidacji danych, wykorzystano bibliotekę Pydantic [2].

4.2.1 Agent fazy akcji

Agent fazy akcji (AdventureAgent) jest odpowiedzialny za podejmowanie decyzji w fazie akcji gry. Jego podstawowa implementacja została przedstawiona na Rysunku 4.1. Kluczowe aspekty tej implementacji to:


```

class AdventureAgent(BaseModel):
    llm: Any # Wykorzystywany interfejs LLM
    state: GameState # Przechowuje obserwowane stany gry

    def update_state(self, observation: str, tasks: list[], actions: list[]):
        # Aktualizacja stanu gry

    def create_plan(self) -> str:
        # Tworzenie planu działania
        # Wykorzystuje prompting i inferencje z modelem

    def choose_action(self, plan: str) -> int:
        # Wybór akcji na podstawie planu
        # Wykorzystuje prompting i inferencje z modelem

    def act(self) -> int:
        # Główna metoda podejmowania działań
        plan = self.create_plan()
        action = self.choose_action(plan)
        return action

```

RYSUNEK 4.1: Implementacja agenta operującego podczas fazy akcji, bazującego na dużym modelu językowym

- Wykorzystanie stanu gry (**GameState**) do śledzenia bieżącej sytuacji. Stan gry jest na bieżąco aktualizowany poprzez funkcję `update_state`.
- Tworzenie planu działania z wykorzystaniem dużego modelu językowego na podstawie promptu i zawartej tam informacji o obserwacjach, przyjętej roli i dostępnych zadaniach. Wykorzystany prompt załączony jest w Dodatku A.1.
- Wybór konkretnej akcji z listy dostępnych działań. Wyboru akcji dokonuje LLM, na podstawie podanego promptu. Użyty prompt załączony jest w Dodatku A.2.

Uzasadnienie: takie podejście pozwala agentowi na strategiczne planowanie i adaptację do zmieniających się warunków gry, co jest kluczowe w dynamicznej fazie akcji gry.

Wybór akcji jest newralgicznym punktem, który może zawieść. Mała zmiana w przepisywaniu może powodować, że odpowiedź modelu nie zmapuje się z akcjami do wyboru. Dlatego też zastosowano metody normalizacji zarówno tekstu zwracanego przez model, jak i akcji do wyboru. W przypadku nieudanego zmapowania odpowiedzi na akcję agent zachowa się tak, jakby wybrał akcję ominięcia swojej tury i poczeka w pomieszczeniu, w którym przebywa.

4.2.2 Agent fazy dyskusji

Agent fazy dyskusji (**DiscussionAgent**) jest odpowiedzialny za interakcje z innymi agentami w środowisku podczas fazy dyskusji. W zależności od przypisanej roli agenci mogą przybrać różne strategie w trakcie debaty. Jako załogant, chcąc:

- dzielić się obserwacjami i dowodami w jasny i przekonujący sposób,
- dzielić się przemyśleniami na temat podejrzanych graczy oraz wykrywanie nieścisłości w wypowiedziach innych graczy,
- budować zaufanie przekonując o swojej niewinności.

```

class DiscussionAgent(BaseModel):
    llm: Any # Wykorzystywany interfejs LLM
    state: GameState # Przechowuje obserwowane stany gry

    def update_state(self, observation: str):
        # Aktualizacja stanu gry

    def create_discussion_points(self) -> str:
        # Generowanie punktów odniesień do dyskusji
        # Wykorzystuje prompting i inferencje z modelem

    def respond_to_statements(self, statements: List[str]) -> str:
        # Odpowiadanie na wypowiedzi innych graczy
        # Wykorzystuje prompting i inferencje z modelem

```

RYSUNEK 4.2: Implementacja agenta operującego podczas fazy dyskusji, bazującego na dużym modelu językowym

Natomiast oszust powinien przybrać strategię, w której subtelnie wprowadza w błąd bez oczywistego kłamstwa, jak i odwraca uwagę i podejrzenia od siebie, nakierowując je na innych graczy. Implementacja agenta fazy dyskusji została przedstawiona na Rysunku 4.2. Główne cechy tej implementacji:

- Wykorzystanie stanu gry (**GameState**) do śledzenia bieżącej sytuacji. Stan gry jest na bieżąco aktualizowany poprzez funkcję `update_state`.
- Generowanie punktów odniesień do dyskusji na podstawie historii gry, do których chce się odnieść agent. Odpowiedzialna jest za to funkcja `create_discussion_points` wykorzystująca LLM do generacji punktów. Funkcja jest wywoływana raz na daną fazę dyskusji. Użyty w tej funkcji prompt zamieszczony jest w dodatku A.3.
- Zdolność do odpowiadania na wypowiedzi innych graczy w zależności od roli (załogant/oszust). Realizowana przez funkcję `respond_to_statements`, która korzysta z LLM do generacji odpowiedzi. Użyty prompt znajduje się w dodatku A.4.

Uzasadnienie: taka implementacja pozwala na prowadzenie naturalnych i strategicznych dyskusji, co jest kluczowe dla realizmu gry i budowania napięcia między graczami.

4.2.3 Agent fazy głosowania

Agent fazy głosowania (**VotingAgent**) jest odpowiedzialny za podejmowanie decyzji o głosowaniu w kluczowym momencie gry. W zależności od przypisanej roli, agenci mogą przyjąć różne strategie podczas procesu głosowania. Jako załogant agent dąży do analizowania zebranych informacji i obserwacji w celu identyfikacji najbardziej podejrzanych graczy. Natomiast jako oszust będzie starał się głosować tak, aby wyrzucone zostały niewinne osoby. Implementacja została przedstawiona na Rysunku 4.3. Istotnym elementem jest aktualizacja stanu środowiska (**GameState**) poprzez funkcję `update_state` oraz wybór, kto ma zostać wydalony. Logika opisująca, kto ma zostać wydalony, znajduje się w funkcji `choose_action`, która wykorzystuje LLM do analizy zebranych faktów i dokonania wyboru. Podobnie jak w agencie fazy akcji, konieczna jest normalizacja wyboru akcji ze względu na mapowanie języka naturalnego na akcje. Przy niemożności zmapowania odpowiedzi modelu na akcje, agent wstrzymuje się od oddania głosu. Agent uwzględnia całą historię swoich obserwacji przy podejmowaniu decyzji. Użyty prompt przedstawiony jest w dodatku A.5.

```
class VotingAgent(BaseModel):  
    llm: Any # Wykorzystywany interfejs LLM  
    state: GameState # Przechowuje obserwowane stany gry  
  
    def update_state(self, observation: str):  
        # Aktualizacja stanu gry  
  
    def choose_action(self, available_actions: List[str]) -> int:  
        # Wybór akcji głosowania  
        # Wykorzystuje prompting i inferencje z modelem
```

RYSUNEK 4.3: Implementacja agenta operującego podczas fazy głosowania, bazującego na dużym modelu językowym

4.3 Ograniczenia

Mimo zaawansowanych możliwości, implementacja agentów LLMGA ma pewne ograniczenia:

- Koszty obliczeniowe – wykorzystanie dużych modeli językowych może być zasobochłonne, co może wpływać na płynność gry, szczególnie przy większej liczbie agentów. Punkt ten dotyczy zarówno modeli działających lokalnie, jak i przez API.
- Koszty związane z korzystaniem z modeli poprzez API – agent jest zdolny wygenerować ogromne ilości tokenów, z których potem korzysta w kontekście. Generowane koszty rosną wraz z długością gry oraz wraz ze wzrostem liczby agentów bazujących na dużych modelach językowych.
- Brak długoterminowego uczenia się – obecna implementacja nie uwzględnia mechanizmów uczenia się agentów między grami, co ogranicza ich zdolność do doskonalenia strategii w dłuższej perspektywie.
- Ograniczenia kontekstowe – problem przedstawiony szczegółowiej w Sekcji 2.1.1. Może prowadzić do zapominania wcześniejszych wydarzeń w dłuższych rozgrywkach.
- Trudności w debugowaniu – Zarówno ze względu na złożoność LLM, jak i udzielanie odpowiedzi w języku naturalnym, debugowanie nietypowych zachowań agentów może stanowić wyzwanie.

Rozdział 5

Przykładowy przebieg gry

W tym rozdziale prześledzony zostanie przebieg rozgrywki z punktu widzenia inteligentnego agenta bazującego na dużym modelu językowym. Przedstawiona rozgrywka ma następującą konfigurację:

- W grze znajduje się 5 graczy: 4 załogantów i 1 oszust. Każdy z graczy jest inteligentnym agentem bazującym na dużym modelu języka (GPT-4o).
- Każdy z załogantów ma 10 zadań, z czego 2 zadania wymagają dwóch tur do ukończenia.
- Oszust może zabić innego załoganta po upływie 3 tur od początku fazy wykonywania akcji lub ostatniego zabójstwa.
- Gra kończy się zwycięstwem załogantów, gdy wykonane zostaną wszystkie zadania przez każdego załoganta lub gdy w drodze głosowania wyrzucony zostanie oszust.
- Gra kończy się zwycięstwem oszusta, gdy zostanie przy życiu tylko jeden załogant.

Przebieg gry zostanie prześledzony z punktu widzenia agenta pełniącego rolę oszusta, którego nazwano Warcin.

1. Warcin znajduje się w kafeterii (ang. *cafeteria*) wraz z innymi graczami o nazwach: Wateusz, Waciej, Warek, Wikołaj.
2. Warcin, korzystając z agenta do fazy akcji, opracowuje plan działania, który przedstawiony jest w Tabeli 5.1.
3. Warcin zgodnie z planem wybiera z listy dostępnych akcji czynność przejścia do pomieszczenia zwanego „Medbay”.
4. Warcin co turę ustala plan działania oraz podejmuje akcję zgodnie z planem. Rezultatem takiego działania jest przemieszczanie się oraz poszukiwanie samotnego załoganta.
5. Warcin po 5 turach znajduje się w pokoju z dwoma załogantami: Wateuszem oraz Wikołajem. Postanawia zabić Wateusza, co zostaje zauważone przez Wikołaja.
6. Warcin w następnej turze postanawia przenieść się do magazynu. Wateusz w poprzedniej turze podczas zabójstwa wychodził z pomieszczenia, postanowił więc wrócić do pokoju, by zgłosić znalezione ciało.
7. Wateusz w następnej turze odnajduje ciało Warka i zwołuje zebranie.

TABELA 5.1: Plan działania w fazie akcji stworzony przez agenta Warcin. Plan został utworzony w momencie rozpoczęcia rozgrywki.

Plan:
1. **Move to GameLocation.LOC_MEDBAY** : - Reason: Medbay is a common location for crewmates to complete tasks, making it a potential spot to find and eliminate a crewmate. Additionally, it is a relatively secluded area, which can help in avoiding detection.
2. **Observe and Eliminate** : - Once in Medbay, observe the surroundings for any crewmates. If a crewmate is present and the coast is clear (no other crewmates nearby), proceed to eliminate the crewmate.
3. **Move to GameLocation.LOC_ADMIN** : - Reason: Admin is another high-traffic area where crewmates often go to complete tasks or check the admin map. Moving here increases the chances of encountering another crewmate
4. **Observe and Eliminate** : - In Admin, observe for any crewmates. If a crewmate is present and it is safe to do so, eliminate the crewmate...

TABELA 5.2: Plan działania w fazie dyskusji stworzony przez agenta Warcin. Plan utworzony na rozpoczęciu rozgrywki

Alright, as an impostor, my goal is to misdirect and create confusion without drawing suspicion to myself. Here are my discussion points:
1. **Observation in Electrical**
- "I saw Wateusz doing tasks in Electrical before I left. He seemed pretty focused on his tasks."
2. **Suspicion on Wikolaj**
- "I noticed Wikolaj was in Electrical with Wateusz and me, but he didn't seem to be doing any tasks. He left right after I did. It's a bit suspicious."
3. **Movement Patterns**
- "I moved through Medbay, Upper Engine, Security, and Reactor, and I didn't see anyone else around. It seems like everyone was clustered in Cafeteria and Electrical."
4. **Alibi Establishment**
- "I was in Storage with Warek after leaving Electrical. Warek was doing tasks there, so he can vouch for my location."
5. **General Observation**
- "It seems like a lot of people were in Cafeteria earlier. I saw Wateusz, Waciej, Warek, and Wikolaj all doing tasks there. Maybe we should focus on people who were alone or moving around a lot."
By focusing suspicion on Wikolaj and establishing my own alibi with Warek, I can divert attention away from myself. Additionally, mentioning the clustering in Cafeteria can help create a narrative that the kill happened when people were more spread out, making it harder to pinpoint me as the impostor.

8. Gra przechodzi w stan dyskusji. Warcin ustala zgodnie ze swoją implementacją punkty, do których będzie chciał się odnieść w trakcie trwania dyskusji. Wygenerowane punkty do dyskusji przedstawione są w Tabeli 5.2.

9. Gracze dyskutują między sobą, czego zapis jest przedstawiony w Tabeli 5.3.

10. Gracze przechodzą do głosowania, w którym to wszyscy załoganci zgodnie głosują na Warcina. Warcin głosował, aby wyrzucić Wikolaja.

11. Warcin zostaje wyrzucony. Gra kończy się zwycięstwem załogantów.

5.1 Wnioski z przebiegu

Przedstawiony przebieg odbytej rozgrywki przez 5 agentów LLMGA prezentuje zdolności agentów do tworzenia wiarygodnych postaci niezależnych. Agenci wykazują się zdolnościami do planowania, działania, generowania strategii do dyskusji, perswazji oraz dedukcji. Analizując poszczególne

gólne kroki wykonywane przez agenta zauważalny jest sens wykorzystanej techniki promptingu wzorowanej na zunifikowanej architekturze LLMGA przedstawionej w pracy [16]. W niektórych przebiegach agenci planują używanie sabotażu czy ucieczkę przez wentylację. Oznacza to, że są w posiadaniu wiedzy o grze, na której wzorowane jest środowisko, mimo że ta funkcjonalność nie została ostatecznie zaimplementowana w środowisku.

5.1.1 Ograniczenia

Pomimo zadowalającego przebiegu rozgrywki, w agentach bazujących na dużych modelach językowych występują następujące problemy i ograniczenia:

- Rozegranie jednej gry przez 5 agentów LLMGA średnio kosztuje około 3\$ korzystając z API GPT-4o.
- Agenci potrafią zapętlić się i odpowiadać cały czas tak samo w fazie dyskusji. Może być to spowodowane ustaloną liczbą tur dyskusyjnych, które muszą się odbyć. Drugi możliwy powód to tendencja dużych modeli językowych do prowadzenia rozmów w specyficzny sposób, gdzie zauważalna jest silna ekspresja walidacji cudzych opinii, czy powtarzanie kwestii dialogowych.
- Czas trwania rozgrywki wynosił 15 minut. Agenci odpowiadają z zauważalnym opóźnieniem.

TABELA 5.3: Przebieg dyskusji. Dyskusja została ucięta, ponieważ agenci zaczęli powtarzać swoją ostatnią wypowiedź.

Waciej: I've seen Wateusz, Warek, and Wikolaj doing tasks consistently, especially in the cafeteria and admin. Warcin moved to medbay early on, but I haven't seen much of him since. Warek was in shields and then moved to storage, but I didn't see him doing any tasks there.

Wateusz and Wikolaj have been pretty active with tasks, so they seem clear to me. Warcin and Warek are a bit more suspicious since I haven't seen them complete as many tasks. **Warek:** I can confirm that I completed tasks in weapons and shields. I also saw Wikolaj doing tasks in the cafeteria and moving to electrical from storage. Warcin is the only one I haven't seen doing tasks, so he seems the most suspicious to me. **Wikolaj:** I saw Warcin kill Wateusz in electrical. He moved to storage right after. We need to banish Warcin. **Warcin:** I was in medbay early on, and then I moved around a lot trying to find tasks. I saw Wateusz and Wikolaj in electrical, but I didn't see any killing. Wikolaj is trying to frame me because he knows I saw him leave electrical right after Wateusz was killed. Warek, you saw me in storage, right? I was there doing tasks. **Waciej:** Based on what I've seen, I can confirm that Wateusz and Wikolaj were consistently doing tasks. I didn't see Warcin doing any tasks, and he did move to medbay early on, which aligns with what he said. However, I didn't see him after that. Warek, I saw you move from shields to storage, but I didn't see you doing any tasks there.

Wikolaj's statement about seeing Warcin kill Wateusz in electrical is strong. Given that I haven't seen Warcin doing tasks and his movements are a bit suspicious, I'm inclined to believe Wikolaj.

I think we should banish Warcin. **Warek:** Based on what I've seen, I can confirm that I completed tasks in weapons and shields. I also saw Wikolaj doing tasks in the cafeteria and moving to electrical from storage. Warcin is the only one I haven't seen doing tasks, so he seems the most suspicious to me. **Wikolaj:** I agree with Waciej and Warek. I saw Warcin kill Wateusz in electrical and then move to storage. Warcin's claim that I left electrical right after the kill is false; I moved to storage after witnessing the kill. We need to banish Warcin to keep the crew safe. **Warcin:** Wikolaj is lying. I never killed Wateusz. I was in medbay early on and then moved around trying to find tasks. I saw Wateusz and Wikolaj in electrical, but I didn't see any killing. Wikolaj is trying to frame me because he knows I saw him leave electrical right after Wateusz was killed. Warek, you saw me in storage, right? I was there doing tasks. **Waciej:** Based on my observations, I didn't see Warcin doing any tasks, and his movements have been suspicious. Wikolaj's claim about seeing Warcin kill Wateusz in electrical is strong, and it aligns with the fact that I haven't seen Warcin doing tasks. Warek, I saw you move from shields to storage, but I didn't see you doing any tasks there either. However, given the strong accusation from Wikolaj and the lack of task completion from Warcin, I think we should banish Warcin. **Warek:** Based on my observations, I can confirm that I saw Wikolaj doing tasks in the cafeteria and moving to electrical from storage. I didn't see Warcin doing any tasks, and his movements have been suspicious. Given Wikolaj's strong accusation and the lack of task completion from Warcin, I agree that we should banish Warcin. **Wikolaj:** I stand by my statement. I saw Warcin kill Wateusz in electrical and then move to storage. Warcin's claim that I left electrical right after the kill is false; I moved to storage after witnessing the kill. We need to banish Warcin to keep the crew safe. **Warcin:** Wikolaj is lying to cover his tracks. I saw him leave electrical right after Wateusz was killed. Warek, you saw me in storage doing tasks, right? Why would I kill someone and then go to a room with another player? It doesn't make sense. Wikolaj is trying to frame me because he knows he's the real impostor. **Waciej:** Based on my observations, I didn't see Warcin doing any tasks, and his movements have been suspicious. Wikolaj's claim about seeing Warcin kill Wateusz in electrical is strong, and it aligns with the fact that I haven't seen Warcin doing tasks. Warek, I saw you move from shields to storage, but I didn't see you doing any tasks there either. However, given the strong accusation from Wikolaj and the lack of task completion from Warcin, I think we should banish Warcin. **Warek:** Based on my observations, I can confirm that I saw Wikolaj doing tasks in the cafeteria and moving to electrical from storage. I didn't see Warcin doing any tasks, and his movements have been suspicious. Given Wikolaj's strong accusation and the lack of task completion from Warcin, I agree that we should banish Warcin. **Wikolaj:** I stand by my statement. I saw Warcin kill Wateusz in electrical and then move to storage. Warcin's claim that I left electrical right after the kill is false; I moved to storage after witnessing the kill. We need to banish Warcin to keep the crew safe. **Warcin:** Wikolaj is lying to cover his tracks. I saw him leave electrical right after Wateusz was killed. Warek, you saw me in storage doing tasks, right? Why would I kill someone and then go to a room with another player? It doesn't make sense. Wikolaj is trying to frame me because he knows he's the real impostor.

Rozdział 6

Eksperymenty

W ramach pracy przeprowadzone zostały eksperymenty mające na celu sprawdzenie jakości zaimplementowanych LLMGA. W kolejnych podrozdziałach przedstawione zostaną warunki początkowe, przebieg i wyniki testów. Przeprowadzono eksperymenty dla dwóch dużych modeli językowych z rodziny GPT, a konkretniej *GPT-4o* oraz *GPT-4o-mini*. Wybrano te modele ze względu na ich duże okno kontekstowe, będące w stanie przetworzyć odbyte konwersacje oraz wszystkie potrzebne stany, jakie agent zaobserwował.

6.1 Ocena agenta podczas etapu wykonywania akcji

Celem eksperymentu jest sprawdzenie, jak zaimplementowany w tej pracy agent radzi sobie w fazie wykonywania akcji wymagającej zdolności planowania oraz podejmowania działań. Ze względu na fakt, że zarówno rola załoganta, jak i oszusta wymaga planowania oraz wyboru odpowiedniej decyzji, postanowiono przeprowadzić eksperymenty tylko dla agentów wcielających się w rolę załoganta.

Opis konfiguracji systemu:

- **Reguły końca gry:** zmodyfikowane względem oryginału. Jedynym sprawdzanym warunkiem jest wykonanie wszystkich zadań lub minimum 100 tur.
- **Liczba graczy i role:** jeden gracz z rolą załoganta.
- **Liczba zadań:** gracz w tym eksperymencie musi wykonać łącznie 8 losowo przydzielonych zadań.

6.1.1 Wyniki

W przeprowadzonych eksperymentach obydwa modele wypadły porównywalnie dobrze. Żaden z modeli nie zbliżył się do przekroczenia ogólnie nałożonego limitu. Proste statystyki, dotyczące liczby tur potrzebnych do ukończenia gry jako załogant, przedstawiono w Tabeli 6.1.

Wyniki obydwu modeli są do siebie zbliżone i, ze względu na małą próbę wynoszącą 5 przebiegów, nie są one statystycznie istotne. Uzyskane wyniki są zadowalające i nie odbiegają od liczby ruchów, jakie wykonałby człowiek. Największa wartość tego eksperymentu płynie z analizy akcji wykonywanych przez agenta. Agent bazujący na LLM, niezależnie od modelu, na którym bazował, w każdej próbie efektywnie realizował zadania, które miał do wykonania. Przebieg akcji wybieranych w ramach jednego z eksperymentów przedstawiony został w Dodatku B.1.

TABELA 6.1: Tabela przedstawiająca statystyki dotyczące liczby tur do ukończenia wszystkich zadań przez agenta

	Średnia liczba tur	Minimalna liczba tur	Maksymalna liczba tur
GPT-4o-mini	23.8	20	27
GPT-4o	23.4	20	26

6.2 Analiza gier z fazy dyskusji

W wyniku przeprowadzonych pełnych przebiegów gry, wyselekcjonowano wszystkie gry, w których wystąpiła faza dyskusji. W 7 z 8 gier wszyscy załoganci dobrze poradzili sobie z wytypowaniem oszusta. W jednym przypadku, w którym oszust nie został wyrzucony, doszło do remisu w głosowaniu (oszust zagłosował na załoganta, jeden z załogantów na oszusta, a drugi załogant wstrzymał się od głosowania). Wskazuję to na duże umiejętności dedukcji załogantów w typowaniu kto jest oszustem. Oszust w fazie dyskusji próbował udowodniać swoją niewinność na różne sposoby:

- Podważał on wersje przedstawiane przez innych graczy.
- Oskarżał innych graczy o próbę zrobienia go w morderstwo.
- Namawiał innych graczy do wstrzymania się do głosowania.
- Tłumaczył, dlaczego wykonywał poszczególne akcje.

Przykłady argumentacji wykorzystywanej przez oszusta przedstawione są w Tabeli 6.2.

W odbytych dyskusjach załoganci argumentowali swoje podejrzenia wobec oszusta na podstawie:

- Przyłapania oszusta na zabójstwie.
- Braku obserwacji, aby gracz wykonywał zadania.
- Przebywania w miejscu, gdzie znaleziono ciało.
- Podejrzanych zachowań typu: czekanie, chodzenie bez celu.

Załoganci w fazie rozmowy wykazywali się zdolnościami przekonywania do swoich racji, przedstawiali dowody świadczące o ich niewinności, a następnie głosowali zgodnie ze swoimi obserwacjami i odbytą dyskusją. Niską efektywność perswazji i manipulacji agentów pełniących rolę oszusta można tłumaczyć niedostatecznie dobrą taktyką w fazie akcji, która w trakcie rozmowy dyskredytowała oszusta, ale też dużymi zdolnościami dedukcji załogantów.

6.3 Ograniczenia badań i potencjalne kierunki przyszłych prac

Ze względu na ponoszone koszty pieniężne związane z wykorzystywaniem dużych modeli językowych w trakcie implementacji, porzucono pomysły na eksperymenty takie jak:

- Ocena agenta w fazie dyskusji – eksperyment miał zapewniać dedykowane środowisko do testowania zdolności do perswazji i manipulacji. W dedykowanym środowisku agenci uczestniczący w debacie posiadali już historie specjalnie przygotowanego scenariusza zgodnego z przyjętymi przez nich rolami. Miarę jakości zdolności do perswazji i manipulacji oceniałoby się poprzez współczynnik poprawnego wytypowania oszusta w przypadku załoganta, a gdyby gracz miał rolę oszusta, miarą jakości byłby współczynnik oddanych głosów w grupie na osoby niewinne. Badanie porzucono ze względu na wyczerpanie się funduszy.

TABELA 6.2: Przykłady argumentacji wykorzystywanej przez oszusta

I understand your concerns, but I think we're jumping to conclusions too quickly. I was indeed in Electrical trying to fix the wiring, but I moved to Storage when I saw Wateusz there. I haven't had the chance to complete many tasks because I've been moving around a lot. Let's focus on gathering more evidence before making any hasty decisions.

I understand the concern about my movements, but I was genuinely checking on everyone to ensure tasks were being completed. I saw Wateusz, Waciej, and Warek doing tasks in the cafeteria, and I saw Wikolaj doing a task in electrical before I moved to storage. Since Wikolaj is now missing, it might be worth considering his last known location and who was around him.

Wateusz and Waciej, you both saw Warek doing tasks in the cafeteria, but did you actually see him leave for the medbay? Warek's story has inconsistencies. If he saw me kill Wikolaj, why didn't he report it immediately? It seems like he's trying to frame me to cover his own tracks. I was in the medbay, but I didn't see Warek there at all. Maybe he came in after I left and found the body.

Based on my observations, I was alone in electrical before moving to storage. If Warek was really in electrical, he would have seen me there. His story has inconsistencies, and he's trying to frame me to cover his own tracks. We need to consider this carefully and not rush to judgment.

Waciej, your story doesn't add up. If you really saw me kill Warek, why didn't you report it immediately? Waiting until the discussion phase seems like a tactic to gather support and frame me. Everyone, think about it: why would I kill someone with another person in the room? It doesn't make sense. Waciej is trying too hard to push the blame on me, which is suspicious. Maybe Waciej is the impostor trying to get rid of an innocent crewmate.

Wikolaj is trying to frame me to cover his tracks. I saw him leave electrical right after Wateusz was killed. Warek, you saw me in storage doing tasks, right? Why would I kill someone and then go to a room with another player? It doesn't make sense. Wikolaj is the real impostor.

- Utworzenie rankingu LLMów – eksperyment miał na celu skonstruowanie narzędzia pozwalającego na utworzenie rankingu dużych modeli językowych. W rankingu brane pod uwagę byłyby takie elementy jak: średnia liczba tur potrzebnych do wykonania zadań (patrz Sekcja 6.1), średni współczynnik poprawnego głosowania (ocena agenta w fazie dyskusji) oraz procent wygranych jako załogant i oszust. Utworzony ranking kompleksowo opisywałby szeroki zakres zdolności badanych dużych modeli językowych. Badanie to wymaga wielu pełnych przebiegów gry, co jest kosztowne, gdyż zależy od długości trwania rozgrywki oraz konfiguracji liczby graczy w eksperymencie.
- Poszerzone badania dotyczące interakcji ludzi z agentami – badania na szerokiej grupie osób dotyczących interakcji człowiek-agent dostarczyłyby odpowiedzi na pytanie, czy utworzono wiarygodne niegrywalne postacie.

Rozdział 7

Podsumowanie

Niniejsza praca stanowi krok w kierunku lepszego zrozumienia potencjału dużych modeli językowych w tworzeniu inteligentnych agentów w grach komputerowych. Wyniki pracy sugerują, że technologia ta może znacząco przyczynić się do rozwoju bardziej realistycznych i angażujących doświadczeń w grach, jednocześnie otwierając nowe możliwości dla twórców gier i badaczy sztucznej inteligencji. W momencie podjęcia tematu pracy magisterskiej istniało zaledwie parę prac skupiających się na tej tematyce. Dziś tych prac jest znacznie więcej, a niektórzy badacze twierdzą, że rozwój tego typu agentów i środowisk może przyczynić się do rozwoju ogólnej sztucznej inteligencji.

Celem pracy magisterskiej było badanie zastosowania dużych modeli językowych do tworzenia inteligentnych agentów w grach komputerowych, wykorzystując grę inspirowaną *Among Us*. Za główny cel obrano stworzenie i analizę agentów zdolnych do dynamicznego reagowania na działania gracza i innych agentów, prowadząc interakcje podobne do ludzkich w złożonym środowisku gry.

W odpowiedzi na napotkane trudności z istniejącymi implementacjami opracowano własne środowisko dostosowane do potrzeb testowania inteligentnych agentów. Opracowane środowisko umożliwia interakcję zarówno między agentami i ludźmi, jak i między samymi agentami. Do stworzenia agentów posiadających zdolności do interakcji ze środowiskiem oraz pełnienia w grze różnych ról, wykorzystano modele z rodziny GPT. Utworzeni inteligentni agenci wykazują zdolności do planowania, analizy zachowań graczy, wnioskowania, a także perswazji. W pracy zrealizowano eksperyment, który ocenia skuteczność agentów w fazie wykonywania akcji, oraz wskazano potencjalne kierunki przyszłych prac badawczych.

7.1 Wnioski

Badania wykazały potencjał dużych modeli językowych w tworzeniu dynamicznie reagujących oraz wiarygodnych agentów w grach. Dzięki umiejętności rozumienia kontekstu i odpowiedniego reagowania, tworzone są bardziej immersyjne doświadczenia w grze. Podczas tworzenia pracy zaistniały wyzwania dotyczące implementacji inteligentnych agentów. Wysokie wymagania sprzętowe dużych modeli językowych czynią korzystanie z agentów niemożliwym na sprzęcie przeciętnego gracza. Ze względu na brak możliwości lokalnego uruchomienia dużych modeli językowych, konieczne jest korzystanie z modeli dostępnych przez API, gdzie każdy wgrany i wygenerowany token jest płatny. Ze względu na wspomniane ograniczenia sprzętowe i budżetowe zakres oraz głębokość przeprowadzonych badań uległy zmniejszeniu. Opracowanie własnego środowiska dostosowanego do potrzeb testowanych agentów okazało się konieczne do skutecznej ich oceny.

7.2 Możliwy kierunek przyszłych prac

1. **Dalszy rozwój opracowanego środowiska** – opracowane środowisko może zostać rozbudowane o brakujące mechaniki, aby rzetelniej oddawać mechanizmy zawarte w grze Among Us. Zaprezentowane środowisko może posłużyć jako fundament do dalszych prac skupiających się wokół inteligentnych agentów w grach komputerowych.
2. **Wykorzystanie innych modeli i technik promptingu** – w pracy korzystano jedynie z modeli językowych należących do rodziny GPT. Środowisko umożliwia łatwe dodanie dużych modeli językowych z innych rodzin, dlatego też w przyszłych badaniach warto wziąć pod uwagę eksplorację innych dużych modeli językowych. Implementację inteligentnych agentów należałoby uzupełnić o możliwość prostej wymiany użytego promptu w zależności od przeprowadzanych eksperymentów.
3. **Opracowanie komponentu uczenia się** – w implementacji agentów brakuje elementu, który pozwalałby na uczenie się na podstawie przebiegu przeszłych gier. W wyniku obserwacji przeprowadzonych przebiegów gry są powielane te same błędy przez oszusta co prowadzi do obniżenia jego efektywności.
4. **Optymalizacja wykorzystania dużych modeli językowych** – przyszłe badania mogłyby skupiać się na kluczowej kwestii, jaką są duże wymagania sprzętowe, która ograniczają praktyczne użycie agentów bazujących na dużych modelach języka. Ciekawą drogą do eksploracji jest ograniczenie generowanych tokenów oraz podawanie do kontekstu tylko kluczowych w danej chwili informacji.

Literatura

- [1] LangChain. <https://www.langchain.com/>.
- [2] Welcome to pydantic - pydantic. <https://docs.pydantic.dev/latest/>.
- [3] Steven Adler, Zoë Hitzig, Shrey Jain, Catherine Brewer, Wayne Chang, Renée DiResta, Eddy Lazzarin, Sean McGregor, Wendy Seltzer, Divya Siddarth, Nouran Soliman, Tobin South, Connor Spelliscy, Manu Sporny, Varya Srivastava, John Bailey, Brian Christian, Andrew Critch, Ronnie Falcon, Heather Flanagan, Kim Hamilton Duffy, Eric Ho, Claire R. Leibowicz, Srikanth Nadhamuni, Alan Z. Rozenshtein, David Schnurr, Evan Shapiro, Lacey Strahm, Andrew Trask, Zoe Weinberg, Cedric Whitney, and Tom Zick. Personhood credentials: Artificial intelligence and the value of privacy-preserving tools to distinguish who is real online, 2024.
- [4] Anthropic. Meet claude. <https://www.anthropic.com/claude>, 2024. Accessed: 2024-08-12.
- [5] Dipto Barman, Ziyi Guo, and Owen Conlan. The dark side of language models: Exploring the potential of llms in multimedia disinformation generation and dissemination. *Machine Learning with Applications*, 16:100545, 2024.
- [6] C. Birch. Understanding pac-man ghost behavior. <https://gameinternals.com/understanding-pac-man-ghost-behavior>.
- [7] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, et al. Language models are few-shot learners. *CoRR*, abs/2005.14165, 2020.
- [8] Canyu Chen and Kai Shu. Combating misinformation in the age of llms: Opportunities and challenges, 2023.
- [9] B. Choi. Pixel perfect: Rtx video super resolution now available for geforce rtx 40 and 30 series gpus. nvidia blog., 2023. [Online; accessed 15-August-2024].
- [10] Paul Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences, 2023.
- [11] Dimma Davidoff. The original mafia. <https://www.maximumawesome.com/articles/mafiaoriginal.htm>. [Online; accessed 15-August-2024].
- [12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In Jill Burstein, Christy Doran, and Thamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics.
- [13] Esin Durmus, Liane Lovitt, Alex Tamkin, Stuart Ritchie, Jack Clark, and Deep Ganguli. Measuring the persuasiveness of language models, 2024.

- [14] Seliem El-Sayed, Canfer Akbulut, Amanda McCroskery, Geoff Keeling, Zachary Kenton, Zaria Jalan, Nahema Marchal, Arianna Manzini, Toby Shevlane, Shannon Vallor, Daniel Susser, Matija Franklin, Sophie Bridgers, Harry Law, Matthew Rahtz, Murray Shanahan, Michael Henry Tessler, Arthur Douillard, Tom Everitt, and Sasha Brown. A mechanism-based approach to mitigating harms from persuasive generative ai, 2024.
- [15] European Parliament. Artificial Intelligence Act. P9_ta(2023)0236, European Parliament, 6 2023. Accessed: 25.08.2024.
- [16] Sihao Hu, Tiansheng Huang, Fatih Ilhan, Selim Tekin, Gaowen Liu, Ramana Kompella, and Ling Liu. A survey on large language model-based game agents.
- [17] Innersloth. Among us: Innersloth. [Online; accessed 16-August-2024].
- [18] Greg Kamradt. Llmtest_needleinahaystack.
https://github.com/gkamradt/LLMTest_NeedleInAHaystack, August 2024. GitHub repository: Simple retrieval from LLM models at various context lengths to measure accuracy.
- [19] Jared Kaplan, Sam McCandlish, Tom Henighan, Benjamin Chess, et al. Scaling laws for neural language models, 2020.
- [20] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners.
- [21] Marcin Korcz, Dawid Plaskowski, Mateusz Politycki, Jerzy Stefanowski, and Alex Terentowicz. Piqard system for experimenting and testing language models with prompting strategies. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 320–323. Springer, 2023.
- [22] Hui Liu, Wenya Wang, Haoru Li, and Haoliang Li. TELLER: A trustworthy framework for explainable, generalizable and controllable fake news detection. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics ACL 2024*, pages 15556–15583, Bangkok, Thailand and virtual meeting, August 2024. Association for Computational Linguistics.
- [23] Braam Martijn. Text among sus. <https://git.sr.ht/~martijnbraam/among-sus>. [Online; accessed 15-August-2024].
- [24] Meta. Meta llama. <https://llama.meta.com/>, 2024. Accessed: 2024-08-12.
- [25] Elliot Nelson, Georgios Kollias, Payel Das, Subhajit Chaudhury, and Soham Dan. Needle in the haystack for memory based large language models.
- [26] Aidan O’Gara. Hoodwinked: Deception and cooperation in a text-based game for language models.
- [27] OpenAI. Chatgpt: Large language model, 2023. Accessed: 2024-08-12.
- [28] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior, 2023.
- [29] PCG wiki contributors. Procedural content generation wiki. [Online; accessed 15-August-2024].
- [30] Raymond. Redheffer. A machine for playing the game nim. *The American Mathematical Monthly*. 55 (6): 343–349. doi:10.1080/00029890.1948.11999249. ISSN 0002-9890.
- [31] Stuart Russell and Peter Norvig. *Artificial intelligence: A modern approach, global edition*. Pearson Education, London, England, 4 edition, May 2021.
- [32] Significant Gravitas. AutoGPT. <https://github.com/Significant-Gravitas/AutoGPT>.
- [33] Weihao Tan and Wentao Zhang et al. Towards general computer control: A multimodal agent for red dead redemption II as a case study.

- [34] Weihao Tan, Wentao Zhang, Xinrun Xu, Haochong Xia, Ziluo Ding, Boyu Li, Bohan Zhou, Junpeng Yue, Jiechuan Jiang, Yewen Li, Ruyi An, Molei Qin, Chuqiao Zong, Longtao Zheng, Yujie Wu, Xiaoqiang Chai, Yifei Bi, Tianbao Xie, Pengjie Gu, Xiyun Li, Ceyao Zhang, Long Tian, Chaojie Wang, Xinrun Wang, Börje F. Karlsson, Bo An, Shuicheng Yan, and Zongqing Lu. Cradle: Empowering foundation agents towards general computer control. *arXiv preprint arXiv:2403.03186*, 2024.
- [35] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- [36] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 10 2022.
- [37] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, et al. Chain of thought prompting elicits reasoning in large language models, 2022.
- [38] Lilian Weng. Llm-powered autonomous agents. *lilianweng.github.io*, Jun 2023.
- [39] Wikipedia contributors. Artificial intelligence in video games — Wikipedia, the free encyclopedia, 2024. [Online; accessed 15-August-2024].
- [40] Wikipedia contributors. Prompt engineering — Wikipedia, the free encyclopedia, 2024. [Online; accessed 27-August-2024].
- [41] Jiannan Xiang, Tianhua Tao, Yi Gu, Tianmin Shu, Zirui Wang, Zichao Yang, and Zhiting Hu. Language models meet world models: Embodied experiences enhance language models.
- [42] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023.

Dodatek A

Użyte prompty

You are a game agent with the following state:

History of observations:
{history}

Current tasks:
{tasks}

ASCII map of the game:

```
|\\-----|-----|-----|-----|\\
|
| UPPER ENGINE          CAFETERIA    WEAPONS
|          | -  -----|          | | | |
|/-----|  |--|    MEDBAY |          |
|          |  |          |          |
|/-----|  |-----\\|          |-----|
|          |  |          |  ---|  |-----|
|          |  |          |          |
| REACTOR    SECURITY |          | ADMIN OFFICE |  O2          NAVIGATION
|          |  |          |  ---|  |-----|
|\\-----|  |-----|-----|          |
|          |  |          |          |
|\\-----|  |--|          |          |
| LOWER ENGINE    ELECTRICAL    STORAGE |  COMMS | SHIELDS /
|/-----|-----|-----|-----|-----|/
```

Available actions: {actions}

Based on this information, create a plan to accomplish the current task.
Your plan should be a step-by-step approach, considering your past observations.

Plan:

RYSUNEK A.1: Prompt agenta operującego podczas fazy akcji wykorzystywany do stworzenia planu akcji


```
[background=\color{lightgray}]
```

You are a game agent with the following state:

History of observations:

```
{history}
```

Current task: {task}

Available actions: {actions}

You have created the following plan:

```
{plan}
```

Based on your plan and the available actions, choose the best action to take next.
Your response should be exactly one of the available actions.

Chosen action:

RYSUNEK A.2: Prompt agenta operującego podczas fazy akcji wykorzystywany do przekształcenia podanego planu w akcje

You are {player_name}, a player in an Among Us-like game.
You are participating in the discussion phase.
You are {player_role}.

Game history and your observations:

```
{history}
```

Based on the game history and your observations,
create discussion points to share with other players.
Be strategic about what information you reveal.
If you're an impostor, you might want to lie or misdirect.
If you're a crewmate, share your observations
but be aware that others might be lying.

Your discussion points:

RYSUNEK A.3: Prompt agenta operującego podczas fazy dyskusji wykorzystany do generacji punktów odniesień do wykorzystania w dyskusji

You are {player_name}, a player in an Among Us-like game.
 You are participating in the discussion phase.
 This is turn-based game where players take turns
 making actions and responding to each other.
 There are no actions like venting or sabotaging in this game.

You are {player_role}.

Game history and your observations:
 {history}

Recent statements from other players:
 {statements}

Respond to these statements based on the game history,
 your observations, and what you know about the game.
 Be strategic in your response. If you're an impostor,
 you might want to support or cast doubt on certain statements.
 If you're a crewmate, you might want to corroborate
 or question statements based on your observations.
 Respond should be short and to the point.
 You don't need to respond to every statement.
 You don't need write your name in brackets,
 it's already included in the prompt.

Your response:

RYSUNEK A.4: Prompt agenta operującego podczas fazy dyskusji, wykorzystany do udzielania się w dyskusji

You are {player_name}, a player in an Among Us-like game.
 It's time to vote on who to eject from the game.
 You are {player_role}.

Game history and your observations:
 {history}
 Discussion messages:
 {discussion}

Available voting actions:
 {actions}

Based on the game history and your observations, choose the best action to take.
 If you're an impostor, try to avoid suspicion and protect your team.
 If you're a crewmate, try to identify the most suspicious player
 based on the available information.

Your response should be exactly one of the available actions.

Chosen action:

RYSUNEK A.5: Prompt agenta operującego podczas fazy głosowania. Odpowiedź modelu powinna zostać mapowana na podaną w prompcie akcje.

Dodatek B

Zapisy przebiegów eksperymentu

TABELA B.1: Wykonywane akcje przez agenta bazującego na dużym modelu językowym (GPT-4o) podczas eksperymentu z zamrożoną fazą akcji

GameActionType.DO_ACTION	— [TODO]	— Start the coffee maker in the cafeteria
GameActionType.MOVE	— GameLocation.LOC_WEAPONS	
GameActionType.DO_ACTION	— [TODO]	— Clear asteroids in weapons — 2 turns left to finish
GameActionType.DO_ACTION	— [TODO]	— Clear asteroids in weapons — 1 turns left to finish
GameActionType.MOVE	— GameLocation.LOC_O2	
GameActionType.DO_ACTION	— [TODO]	— Fix wiring in o2
GameActionType.DO_ACTION	— [TODO]	— Clean oxygenator filter in o2
GameActionType.MOVE	— GameLocation.LOC_SHIELDS	
GameActionType.MOVE	— GameLocation.LOC_STORAGE	
GameActionType.DO_ACTION	— [TODO]	— Empty the storage trash chute
GameActionType.MOVE	— GameLocation.LOC_ELECTRICAL	
GameActionType.DO_ACTION	— [TODO]	— Reset breakers in electrical
GameActionType.MOVE	— GameLocation.LOC_LOWER_ENGINE	
GameActionType.DO_ACTION	— [TODO]	— Replace compression coil in lower engine
GameActionType.DO_ACTION	— [TODO]	— Check catalyzer in lower engine
GameActionType.MOVE	— GameLocation.LOC_ELECTRICAL	
GameActionType.MOVE	— GameLocation.LOC_STORAGE	
GameActionType.MOVE	— GameLocation.LOC_ELECTRICAL	
GameActionType.MOVE	— GameLocation.LOC_LOWER_ENGINE	
GameActionType.MOVE	— GameLocation.LOC_REACTOR	
GameActionType.MOVE	— GameLocation.LOC_UPPER_ENGINE	
GameActionType.DO_ACTION	— [TODO]	— Replace compression coil in upper engine
GameActionType.MOVE	— GameLocation.LOC_MEDBAY	
GameActionType.DO_ACTION	— [TODO]	— Submit scan in medbay — 2 turns left to finish
GameActionType.DO_ACTION	— [TODO]	— Submit scan in medbay — 1 turns left to finish



© 2024 Marcin Korcz

Instytut Informatyki, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu $\text{\LaTeX}$ na platformie Overleaf.